Priroda User Manual

Priroda manual contributors

Oct 17, 2026

PART I. INTRODUCTION

1 About Priroda **2**
- 1.1 Main features . . . 2
- 1.2 Capability matrix . . . 3
- 1.3 What Priroda does not do . . . 4
- 1.4 Version history . . . 4
- 1.5 Obtaining Priroda . . . 5
- 1.6 How to cite . . . 6

2 Installing and running **7**
- 2.1 Executables . . . 7
- 2.2 Running a calculation . . . 8
- 2.3 Memory, disk and scratch: `$system` . . . 10
- 2.4 Parallel performance . . . 11
- 2.5 Batch systems . . . 12
- 2.6 Input file requirements . . . 13
- 2.7 Basis-set files . . . 14

3 Getting started **15**
- 3.1 Before you start . . . 15
- 3.2 A single-point energy . . . 15
- 3.3 Optimizing a geometry and confirming a minimum . . . 17
- 3.4 Reading the output . . . 22
- 3.5 Where next . . . 23

4 Input syntax **24**
- 4.1 Groups . . . 24
- 4.2 Keywords . . . 25
- 4.3 Values . . . 25
- 4.4 The `$molecule` group . . . 26
- 4.5 Text outside groups . . . 27
- 4.6 Appending results of earlier runs . . . 27
- 4.7 Common mistakes . . . 28

5 Input reference **30**
- 5.1 `$system` . . . 30
- 5.2 `$control` . . . 32
- 5.3 `$molecule` . . . 36
- 5.4 `$guess` . . . 38
- 5.5 `$scf` . . . 39
- 5.6 `$dft` . . . 41

5.7 $grid . . . 42
5.8 $integral . . . 43
5.9 $atoms . . . 44
5.10 $optimize . . . 44
5.11 $d2edr2 . . . 49
5.12 $thermo . . . 51
5.13 $nmr . . . 51
5.14 $Energy . . . 52
5.15 $dynamic . . . 54
5.16 $uniq . . . 55

6 Solving common problems **56**
6.1 The input is not read, or is read wrongly . . . 56
6.2 SCF does not converge or diverges . . . 58
6.3 Geometry optimization problems . . . 60
6.4 Wrong or suspicious results . . . 61
6.5 Memory, disk and scratch errors . . . 63
6.6 Parallel and MPI problems . . . 64

7 Sources **67**
7.1 Program files and text from the developer . . . 67
7.2 User-written material . . . 67
7.3 Published papers . . . 68

Index **69**

An unofficial user manual for Priroda, the quantum-chemistry program by Dimitri Laikov.

Important

This manual is not an official Priroda document. It was not written or reviewed by the developer. It is compiled from the program's own files, the short keyword list that ships with it, and user reports from forums and user pages; *Sources* lists them all. When this manual and the program disagree, the program is right.

Warning

Work in progress. Unless an entry says otherwise, nothing here has yet been checked against a running Priroda binary. Each keyword entry gives its sources and its verification status.

CHAPTER

ONE

ABOUT PRIRODA

Priroda (Russian for "nature") is a quantum-chemistry program for molecules written by Dimitri N. Laikov. It began as a fast program for GGA density-functional calculations and grew to cover hybrid DFT, RI-MP2, coupled-cluster methods, TDDFT, a scalar-relativistic Hamiltonian, NMR, and Laikov's own semiempirical model.

The program is closed-source and has no complete official manual. Priroda's own documentation is a short keyword list, the "Input Summary" (`input.txt`), which ships with the program. This manual collects that text, the example files on the official site, and twenty years of user experience from Russian-language forums. It is not an official document: the developer has not written or reviewed it. *Sources* lists what it is based on.

1.1 Main features

Density-functional theory
: GGA DFT in which the Coulomb and exchange–correlation terms are evaluated with an auxiliary (fitting) basis. This is the source of Priroda's speed. Every orbital basis comes with a matched fitting basis. GGA DFT has analytic second derivatives.

  Hybrid functionals with exact exchange evaluated by the resolution of the identity, under *theory=ridft*.

Correlated methods
: RI-MP2 and, as undocumented or less-used options, conventional MP2–MP4, CCSD and CCSD(T). Gradients exist for MP2, CCSD and CCSD(T).

Excited states
: GGA TDDFT with excited-state gradients, so excited-state geometries can be optimized.

Relativistic calculations
: A scalar-relativistic Hamiltonian (*four=1*) with all-electron basis sets and a finite-nucleus model. The user FAQ of one computing centre names this, together with basis sets for the whole periodic table, as what sets Priroda apart. Effective core potentials (SBK) are also available.

Geometry and dynamics
: Minimization in redundant internal coordinates, transition-state search, IRC, relaxed scans, Hessians with vibrational analysis and thermochemistry, and molecular dynamics with a thermostat.

Properties
: NMR shielding tensors and chemical shifts (also at points in space, for NICS), Hirshfeld and Mulliken charges, bond orders, ESR parameters, critical points of the electron density, Raman intensities, and orbitals and normal modes for Molden.

Semiempirical model
: Laikov's parametrized "QM" model (*theory=qm_n3*), used for systems of thousands of atoms.

Parallel execution
: MPI parallelism. In practice it scales well within one node and poorly beyond it; see *Installing and running*.

Experienced users report that Priroda is markedly faster than general-purpose programs for routine GGA work. One 2018 comparison found it about three times faster per optimization step than ORCA 4.0 (PBE/cc-pVDZ, one core).

1.2 Capability matrix

The table shows which `task=` values are known to work with each `theory=` value. It is assembled from example inputs and user reports, not from a statement by the developer.

- ✓: shown to work in an example input or a user report.
- –: not available, according to the sources.
- ?: unverified. No source shows it working or failing.

Task	`dft`	`ridft`	`tddft`	`rimp2`	`ccsd` / `ccsd4t`	`qm_n3`
`energy`	✓	✓	✓	✓	✓	✓
`gradient`	✓	✓	✓	✓	✓ (1)	✓
`hessian`	✓ (2)	?	?	?	✓ (3)	?
`optimize`	✓	✓	✓ (4)	✓	✓ (1)	✓
TS (`saddle=1`)	✓	?	?	?	?	?
`irc`	✓	?	?	?	?	?
`scan`	✓	?	?	?	?	?
`nmr`	✓ (5)	–	–	–	–	–
TDDFT excited states	–	– (6)	✓	–	– (7)	–
MD (`thermostat`)	?	?	?	?	?	✓

Notes to the table:

1. A CCSD optimization is shown in a user input. CCSD(T) gradients are implied by the developer's 2011 note on a bug fix that names the CCSD(T) gradient.
2. Analytic second derivatives.
3. Users report CCSD and CCSD(T) Hessians. For correlated methods they are computed by finite differences of the gradient, which takes many gradient calculations.
4. Excited-state optimization works, but Priroda has no root following, so the state can switch during the optimization.
5. NMR works only with `theory=dft` and a non-hybrid functional. Users report that relativistic (`four=1`) NMR runs hang.
6. There is no hybrid TDDFT. See *What Priroda does not do*.
7. Excited states at the coupled-cluster level use a different `theory=` value, `ccsdsd` (linear-response CCSD, RHF only), whose status is undocumented.

The scalar-relativistic option `four=1` has been used with `dft`, `ridft` and `rimp2`. The full list of `theory=` values, including the undocumented ones, is in *$control theory*.

Warning

Hybrid functionals work only with `theory=ridft`. With `theory=dft` or `theory=tddft`, older versions silently drop the exact exchange and run what amounts to a pure GGA, and newer versions abort. See *$control theory*.

1.3 What Priroda does not do

Check this list before planning a project around Priroda.

No use of point-group symmetry in older versions
Older versions always run in C1. A symmetric starting structure stays symmetric during optimization, so a Hessian is needed to show whether it is a minimum. Newer outputs (v13 onwards) print a detected point group, and the official 2023–2025 example inputs use `sym=1` in `$molecule` and `$optimize`. What these options do is not documented; see *$molecule sym*.

No continuum solvation
No version has a continuum solvent model. Solvent can only be included as explicit molecules.

No D3 or other empirical dispersion correction
The developer has declined to add one (relayed from correspondence). Users add Grimme's D3 a posteriori with the external `dftd3` program. The 2023–2025 functionals `LPBEVV` and `LPBEVVV` may include dispersion, but this is inferred from their test systems and not documented.

No periodic systems
Priroda handles molecules and clusters only.

No dummy atoms, so no counterpoise (BSSE) correction
Ghost centres cannot be placed in `$molecule`. For NICS, give the probe points in *$nmr points* instead.

No hybrid TDDFT
TDDFT is GGA only. A hybrid functional with `theory=tddft` does not give hybrid TDDFT.

Other gaps reported by users:

- no spin–orbit coupling, so no phosphorescence calculations;
- no multireference methods (CASSCF, MRCI, CASPT2) that users could get working;
- no freezing of atoms in Cartesian space; only internal coordinates can be fixed (*$optimize fix*);
- no general job chaining in one input: `task=optimize+hessian` is the only combined task, and two `task=` lines run only the first.

1.4 Version history

Priroda numbers its versions with integers and prints the build date in the output banner, for example `Priroda 13 (2013.01.10)`. Energies differ slightly between versions, so keep every calculation you compare on one version.

Version	Build dates seen	Notes
2	2002.01.14 (v2.10)	Banner `program Priroda version 2.10`. Had `task=optimize+hessian`.
4	2004.09.14 (v4.07)	The finite-nucleus model changed between versions 3 and 4, so pre-v4 relativistic basis sets must not be used with v4 and later.
5	—	Version described by the "Input Summary" mirrored on the KNC wiki. Introduced the systematic L basis sets. Hybrid RI-DFT was implemented in November 2004.
6	2006.08.20	Long the most widely used version: 32- and 64-bit Linux builds with MPI, and a Windows build (`p6.exe`). `optimize+hessian` was removed. Parallel Hessians use at most 2 cores.
7	—	Sent as object files for linking against a local MPI library.
8	2008.09.05	Built on request for a Myrinet cluster.
9	2009–2011	`optimize+hessian` returned. In mid-2011 the developer fixed a long-standing bug that in rare cases gave wrong gradients for MP2, MP4, CCSD, CCSD(T), CIS(2) and EOM-CCSD.
10	2011.04.14	Linux build `p` with a bundled `mpiexec`.
11	—	Contains the semiempirical QM model.
13	2013.01.10, 2013.01.21, 2013.09.13	Builds behind the example outputs on the official site.
14	2014.01.01	Input geometry echoed with a < prefix (<`$molecule`). 8060-atom QM-model run on 32 processes.
15	2015.09.14	Scalar-relativistic example (Pt complex).
16, 17	2016–2018	The Windows build runs in parallel from version 16. Version 17 runs Hessians on 4 or more cores. Around 2016 the developer removed limits on SCF memory allocation and on the system size for Hessians.
23	2023.06.22	Newest numbered version reported by users (2024). New functionals `LPBEVV` and `LPBEVVV`.
?	2025	Linux builds dated 2025-02-28 (`p4_d`) and 2025-04-14 (`p_l`), with 2025 example inputs that use `theory=DFT_n4`. Version number not known.

Version numbers missing from the table (3, 12, 18–22) do not appear in any source. The Russian file `Dft_ansi.txt` describes Priroda's predecessor, which used different group names (`$coordinates`, `$SCF`, `$search`, `$force`). Those names are not part of the current keyword list.

1.5 Obtaining Priroda

Priroda is free of charge and distributed by the developer. The usual route is to write to Dimitri Laikov by e-mail and ask for the program. Users reported the address `dimitri_laikov@mail.ru` from 2010 to 2018; earlier it was `laikov@physto.se`. Users note that he does not always answer quickly.

Current binaries and data files are also listed at https://rad.chem.msu.ru/~laikov/p_w/. The page is a bare file index with no documentation, license text or contact information. It holds:

- Linux x86-64 executables (`p`, `p4`, `p_d`, `p4_d`), builds for particular clusters (`p_l`, `p_mvs1p1`, `p_mvs10p`, `p_mvs10q`), an ARM build (`p_arm`), a bundled `mpiexec`, and the molecule viewer `xm`;
- Windows (Cygwin, x86-64) executables in `exe64/`;
- example inputs and outputs;
- basis-set and parameter files (`basis.in`, `basis4.in`, `b2*.in`, `b4s*.in`, `3z.bas`, `sbk.bas`, `qm.in`);

- small shell scripts: `ge` (append the `$Energy` block from an output to the input), `gm` (replace `$molecule` with the optimized geometry) and `grx` (join two IRC halves).

The executables are described in *Installing and running*.

Note

The legal status is not stated on the official site. A 2009 presentation relayed on a forum said that Priroda is registered with Rospatent and distributed through the company OOO "Dimonta". A user reported in 2018 that the developer had said, a couple of years earlier, that there was no licence agreement. Users customarily ask the developer's permission before using the program.

The official site has hosted builds with known defects. In 2016 it carried a note that the `w64/*.exe` builds (an attempt at 64-bit Cygwin) had a DFT/ECP bug. If a build behaves strangely, ask the developer whether it is the right one.

1.6 How to cite

Priroda has no single program paper for the current version. Users cite the papers below; the first three are the list given by the KNC wiki FAQ and repeated in the forums.

Program:

- D. N. Laikov, Yu. A. Ustynyuk. PRIRODA-04: a quantum-chemical program suite. New possibilities in the study of molecular systems with the application of parallel computing. *Russ. Chem. Bull., Int. Ed.* **54** (2005) 820–826. Russian original: *Izv. Akad. Nauk, Ser. Khim.* 2005, No. 3, 804–810.

Density-fitting DFT (cite for any DFT calculation):

- D. N. Laikov. Fast evaluation of density functional exchange-correlation terms using the expansion of the electron density in auxiliary basis sets. *Chem. Phys. Lett.* **281** (1997) 151–156.

L-family basis sets (`L1`, `L2`, `L11`, ...):

- D. N. Laikov. A new class of atomic basis functions for accurate electronic structure calculations of molecules. *Chem. Phys. Lett.* **416** (2005) 116–120.

The `3z` basis set, which has no journal paper:

- D. N. Laikov. "Development of an economical approach to the density functional calculation of molecules and its application to complex chemical problems". PhD dissertation, Moscow State University, 2000 (in Russian).

Scalar-relativistic DFT (`four=1`):

- D. N. Laikov. An implementation of the scalar relativistic density functional theory for molecular calculations with Gaussian basis sets. DFT2000 Conference, Menton, France, 2000.

Semiempirical QM model (`theory=qm_n*`):

- D. N. Laikov. *J. Chem. Phys.* **135** (2011) 134120.

For version 6, users cited the program itself as "D. N. Laikov, PRIRODA, Electronic Structure Code, Version 6, 2006". A 2014 forum suggestion adds Laikov's home page, http://rad.chem.msu.ru/~laikov/, as the reference for the current version.

Also cite the original papers for the functional, and for the Hamiltonian and nuclear model if you use `four=1`. *$dft functional* and *$control four* give them.

CHAPTER

TWO

INSTALLING AND RUNNING

Priroda has no installer. You copy an executable, the basis-set files you need and, for parallel runs, the MPI launcher into a directory, write an input file, and start the program with the input and output file names as arguments.

2.1 Executables

2.1.1 Linux

The official site lists these Linux files. Sizes and dates are those of the 2025 file index.

File	Date	Description
`p`	2023-06-25	Main x86-64 executable, statically linked.
`p4`	2023-06-25	Build "for FMA/AVX2" processors.
`p_d`	2021-11-12	Dynamically linked build.
`p4_d`	2025-02-28	Dynamically linked FMA/AVX2 build.
`p_l`	2025-04-14	Build for a particular cluster, started through the site's Intel MPI wrapper.
`p_mvs1p1`, `p_mvs10p`, `p_mvs10q`	2019–202	Builds for particular clusters.
`p_arm`, `mpiexec_arm`	2015-09-23	ARM build and its MPI launcher.
`mpiexec`	2012-09-09	MPI launcher shipped with `p`.
`xm`	2025-04-24	The developer's molecule viewer. It takes a Priroda input file as its argument (`xm C60.qm.in`).

That `p` is statically linked is inferred from the description of `p_d` as "dynamically linked". The site does not say which cluster each `p_mvs*` and `p_l` build is for.

Older distributions used other names: `p6`, `p6_32` and `p6_64` for version 6, `p9`, and `p1` for version 11 at one site. Scripts and forum posts written for those versions use these names. This manual writes `p` throughout.

Some users run the Linux build under WSL on Windows.

2.1.2 Windows

The Windows build is a Cygwin build for x86-64. It is in the `exe64/` directory of the official site:

File	Date	Description
`p1.exe`	2021-10-01	Serial executable.
`p1_190306.exe`	2019-03-12	Older serial build.
`p_190306.exe`	2019-03-12	Older parallel (MPI) build.
`xm.exe`	2019-03-08	Viewer.
`cygwin1.dll`	2021-10-01	Cygwin runtime library.

For serial runs you need only `p1.exe` and `cygwin1.dll` in the same directory. A full Cygwin installation is not required, and no administrator rights are needed. For parallel runs see *Parallel runs on Windows*.

Note

In 2017 users referred to the parallel Windows build as `exe64/p.exe`. The current index has no `p.exe`; the newest parallel build there is `p_190306.exe`.

Old 32-bit Windows builds could not allocate more than about 1 GB of memory; `malloc(...) failed` appeared above that. Users report that the 64-bit build has no such limit.

2.2 Running a calculation

2.2.1 Serial runs

Give the input file first and the output file second:

```
p input.in output.out
```

On Windows:

```
p1.exe input.in output.out
```

On Linux the current directory is usually not in `PATH`, so start the program as `./p` or with its full path. If the program cannot find the input or the basis file, give full paths everywhere: for the executable, the input, the output, and the basis file inside the input.

Run in a directory you own and can write to. Keep file and directory names to Latin letters, digits and simple punctuation, with no spaces or Cyrillic. This also applies to the computer name on Windows. File names are case-sensitive on Linux, although keywords are not.

Note

One user saw the complaint `0: output file 'tasK.out'?` when the output file name contained a capital letter. This is not confirmed. Lower-case file names avoid the question.

Older versions also wrote the output to standard output when no output file was given.

2.2.2 Parallel runs on Linux

Priroda is parallelized with MPI. Three command forms appear in the sources, and which one works depends on the build:

```
p -np 4 input.in output.out                  # the executable starts its own processes
./mpiexec -np 4 ./p input.in output.out      # the mpiexec shipped with p
mpiexec -np 4 p input.in                     # as used for version 14 at one site
```

The first form is what users ran with versions 6 and 10. The second is what the official site shows for the current `p` and its bundled `mpiexec`. In one report with version 10, starting through the bundled `mpiexec` failed while `p -np 4` worked, but that run also had a relative basis-file path, which can cause the same failure. Try the form from the official site first.

Check the start of the output. A parallel run prints the number of processes and one line per process:

```
 parallel on  4 processors
 node  0: 'linux-ldmm'
 node  1: 'linux-ldmm'
 ...
```

The cluster builds are started through the cluster's own MPI. The official example for `p_l` under SLURM is:

```
sbatch -p test -n 16 -t 6 impi ~/bin/p_l file.{in,out}
```

Here `impi` is that cluster's wrapper script for Intel MPI, not part of Priroda.

2.2.3 Parallel runs on Windows

The official site shows this command for a two-process run:

```
mpirun.exe -np 2 p.exe C60.qm.in C60.qm.w.2.out
```

The MPI runtime is not on the site. Users assembled it from Cygwin as follows:

1. Download the 64-bit Cygwin installer (`setup-x86_64.exe` from https://cygwin.com). Install the default packages plus the `openmpi` package.
2. Copy these files from the Cygwin installation into the directory with the Priroda executable: `cygwin1.dll`, `cyggcc_s-seh-1.dll`, `cygmpi-12.dll`, `cygopen-pal-13.dll`, `cygopen-rte-12.dll`, `orterun.exe`. Newer OpenMPI packages have `cygopen-pal-40.dll` and `cygopen-rte-40.dll` instead of the `-13`/`-12` files; take those. Take `cygwin1.dll` from the same installation, because the other DLLs work only with matching versions. After this, Cygwin itself is no longer needed.
3. Create a directory `dev` one level above the directory you start Priroda from, with subdirectories `shm` and `mqueue`. For Priroda in `C:\Priroda`, that means `C:\dev\shm` and `C:\dev\mqueue`. Without a Cygwin installation, the Cygwin libraries take the parent of the start directory as the root `/`.
4. If the program is in a system directory such as `C:\Program Files`, run it as administrator.
5. Set `TMPDIR` and start the run from `cmd.exe`:

   ```
   cd /d "C:\Priroda"
   set TMPDIR="%TEMP%"
   orterun.exe -np 2 p.exe "C:\Work\file.in" "C:\Work\file.out"
   ```

Problems reported with this setup:

- The recipe needs a Priroda build that supports parallel runs. Version 6 (`p6.exe`) failed with an OpenMPI `agent-not-found` error; users moved to version 17.
- On a computer connected by Wi-Fi that also has a wired network adapter, OpenMPI stops with `tcp_peer_send_blocking: send() to socket 12 failed: Transport endpoint is not connected (128)`. Disable the wired adapter.
- With the Ethernet cable unplugged, one user's run used only two of the cores.
- `Warning: '/dev/shm' does not exists or is not a directory.` means the Cygwin environment is missing. Create the `dev` directories (step 3), or start from a Cygwin shell.

Warning

A user reported in 2021 that this recipe no longer worked on Windows 10 with a current Cygwin, and no fix was posted. Expect to adapt it to the Cygwin and OpenMPI versions you have.

2.3 Memory, disk and scratch: `$system`

The *$system* group sets the resources. Every limit applies **per MPI process**. `disk=10` on four processes can use up to 40 GB.

```
$system memory=2048 disk=-2048 path=/scratch/user $end
```

memory
: Memory in MB. Two values set rank 0 and the other ranks separately: `mem=2048,256` gives the master process 2048 MB and every other process 256 MB. In a parallel run, most of the memory and all of the disk storage are used by rank 0, so give the other ranks less.

disk
: A positive value is scratch disk in **GB**. The program opens one scratch file per gigabyte. A negative value is scratch kept in memory, in **MB** (a RAM disk): `disk=-2048` holds 2048 MB of scratch in RAM. A second value applies to the other ranks, as for `memory`; the official example inputs typically give them `-1`.

path
: The scratch directory. A colon-separated list gives one directory per process, which pays off only when the directories are on different physical drives. For several processes and one fast disk, a single directory is enough.

2.3.1 Keeping scratch in memory

With a negative `disk`, `memory` plus —`disk`— must fit into the physical RAM for each process, or the run stops at once with `malloc(...) failed`. One user's setting for a 24 GB node is `memory=9000 disk=-14000`. Keeping the scratch in RAM is faster than any disk. In one RI-MP2 benchmark on 8 cores it cut the time from 86 minutes (scratch on disk) to 12.4 minutes.

2.3.2 Where scratch files go

The scratch directory is taken from `path=`, then from the `TMPDIR` environment variable. The files are named `tmp<number>.<rank>`, for example `/tmp/tmp2960.0`. Priroda deletes them at the end of a normal run; after a crash, delete the leftovers yourself.

Note

The fallback when neither `path=` nor `TMPDIR` is set is reported differently. `input.txt` says the current working directory. A 2020 user report says the directory of the input file. Version 6 printed `Disk: 1 GB '/tmp'` when `$system` was omitted. Set `path=` or `TMPDIR` explicitly.

On Windows there is no `/tmp`. Without `path=` or `TMPDIR`, older versions stopped after printing `0: opening file /tmp/tmp2960.0`. Use `$system path=. $end` (the current directory) or set `TMPDIR`.

2.3.3 Disk speed

Warning

Never put scratch files on NFS or another network file system. On most clusters the home directories are on NFS; use the node-local scratch disk instead.

Priroda is sensitive to disk speed. Priroda's own documentation (`input.txt`) warns that for all methods except GGA DFT, performance "will be degraded terribly" if more than one process reads and writes large files on the same physical drive. In order of preference:

1. keep the scratch in RAM (negative `disk`) if it fits;
2. give each process its own physical drive with `path=a:b:c:d`;
3. put all processes on one fast local disk (SSD), accepting some loss.

If the disk is the bottleneck for RI-MP2 or RI-DFT, *$integral direct=1* recomputes the integrals instead of storing them. One user found it uses about a third of the memory but runs about ten times longer in CPU time. In a disk-bound RI-MP2 optimization another user got 20 steps overnight with it instead of 2.

Users also recommend more RAM over faster disks. Scratch on a separate SSD helps, but RAM matters more.

2.3.4 When the limits are too small

Priroda does not fall back to a slower algorithm when memory or scratch runs out. The run stops with an error such as `malloc(536870912) failed` (memory) or `0: out of disk space (VNNNN), 184441541KB` (scratch). The number in the disk message is the amount needed. Raise the limit in `$system`, move to a node with more resources, or choose a cheaper method (RI-MP2 instead of conventional MP2).

If you omit `$system`, version 6 printed `Memory: 128 (128) MB` and `Disk: 1 GB '/tmp'`. Defaults in current versions are not documented, so always give `$system` for anything larger than a test.

If a run fails for lack of memory or disk, see *Memory, disk and scratch errors*.

2.4 Parallel performance

Users consistently report the following:

- **DFT scales well within one node, up to about 6–8 cores.** Beyond 8 cores the gain is at most about 10%, even for large systems. One 38-step optimization took 101, 63 and 37 s on single nodes with 8, 12 and 24 cores. A small PBE/L1 optimization with version 10 took 81.5, 42.5, 29.8 and 29.3 s on 1–4 cores.
- **Running across nodes gives no gain for DFT.** Use one node per job.

- **Memory per process does not drop** when you add processes, and in a parallel Hessian nearly all of the disk use is on rank 0.
- **RI-MP2 parallelizes better than DFT**, at least in memory. The v5 documentation limits useful parallelism to Nproc $\leq$ min(N_atoms, N_occ) and recommends a number of processes that divides the number of occupied orbitals nearly evenly. For N_occ = 14 it rates 2, 7 or 14 processes as best, 3 or 5 as acceptable, 6 as poor and 13 as very bad.
- **Small RI jobs on many processes can hang.** RI-MP2 and RI-DFT hang with no message when a process receives no work. The output then shows a line `0 blocks ... on node N` after `integral estimates`. Run small RI jobs on one or a few cores.
- **Several independent jobs often beat one parallel job.** On a 4-core machine, four single-core jobs ran about 30% faster in total than the same jobs run one after another on 4 cores each.

Results can differ in the last digits between runs on different numbers of processes. An unstable TS search was once seen to follow different paths for different `-np`.

Note

Older versions scaled worse. In version 6 a parallel Hessian could use at most 2 cores and could hang on 4, and a version 7 MP2 run hung on more than about 24 cores. Use version 17 or later for parallel work.

Hangs, crashes and poor speed-up in parallel runs are covered in *Parallel and MPI problems*.

2.5 Batch systems

Priroda needs nothing special from a batch system. The job script has to:

1. request **one node** and the number of cores you will use (see *parallel performance above*);
2. point the scratch to the node's local disk or to RAM, by exporting `TMPDIR` or by `path=` in `$system`;
3. make the memory request cover `memory` + —negative `disk`— for every process;
4. start Priroda with the input and output file names.

A SLURM script along these lines should work. Directory names and module setup are site-specific.

```
#!/bin/bash
#SBATCH --job-name=mol
#SBATCH --nodes=1
#SBATCH --ntasks=8
#SBATCH --time=24:00:00

PRIRODA=$HOME/priroda/p                      # full path to the executable
cd "$SLURM_SUBMIT_DIR"
export TMPDIR=/local/scratch/$SLURM_JOB_ID   # node-local disk; site-specific
mkdir -p "$TMPDIR"

"$PRIRODA" -np "$SLURM_NTASKS" mol.in mol.out
# or, if your build needs it:  ./mpiexec -np "$SLURM_NTASKS" "$PRIRODA" mol.in mol.out

rm -rf "$TMPDIR"
```

Under PBS/Torque the same steps read the number of cores from `$PBS_NODEFILE`. PBS often provides a per-job local scratch directory and deletes it after the job.

```
#!/bin/bash
#PBS -N mol
#PBS -l nodes=1:ppn=8,walltime=24:00:00

cd "$PBS_O_WORKDIR"
NP=$(wc -l < "$PBS_NODEFILE")
export TMPDIR=/local/scratch/$PBS_JOBID      # site-specific
mkdir -p "$TMPDIR"

$HOME/priroda/p -np "$NP" mol.in mol.out

rm -rf "$TMPDIR"
```

If `/tmp` on the node is small or has a quota, a large job dies writing a scratch file, for example `0: write file /tmp/tmp9633.11 (leAX): ...`. Point the scratch at the cluster's scratch file system or at RAM.

Note

Historical: the KNC wrappers. The Kazan Scientific Centre clusters (2006–2012) provided the wrappers `pbspriroda5`, `pbspriroda5-mpi` and `pbspriroda6`, called as `pbspriroda6 file_name -time hh:mm:ss -queue queue -mem MB -disk size -procs 1:ppn=N`. The wrappers wrote the `$system` group themselves from `-mem` and `-disk`, so inputs submitted through them had to omit `$system`. Forum posts and old FAQs that mention these commands refer to that site only.

2.6 Input file requirements

The input is a plain text file. Write it in a plain-text editor, not a word processor.

- **Plain ASCII.** Keywords, values and file names should contain ASCII characters only. Do not save the file as Unicode.
- **Unix line endings (LF).** This holds on Windows too. Saving from Word as "plain text" still leaves Windows (CRLF) line endings, and some editors need to be told explicitly to write Unix line endings.
- **Straight quotes.** Write `basis="3z.bas"` with the ASCII double quote. Typographic quotes, as in `basis=≫3z.bas≫`, break the basis-file name.

An input with CRLF line endings fails in `$molecule`, because the carriage return becomes part of the keyword. The output stops after a line like this, where the keyword `z-matrix` or `cartesian` is shown garbled by the carriage return:

```
 molecule input: ''
' ?matrix
```

Variants seen are `?rtesian` for `cartesian`, `'z-matrix' ?`, and `'cartesian` followed by `' ?` on the next line. A less obvious case: an input edited in Word ran `task=scan` as a plain optimization.

To check and fix the line endings on Linux:

```
file mol.in        # reports "with CRLF line terminators" if the file has Windows endings
dos2unix mol.in
```

Running `dos2unix` on a file that already has Unix line endings does no harm. On Windows, Notepad++ and the FAR Manager editor (Shift+F2, then UNIX (LF)) can convert line endings.

For the full list of input-reading failures and their symptoms, see *The input is not read, or is read wrongly*.

2.7 Basis-set files

The basis sets, their auxiliary (fitting) sets and the ECPs are read from a separate file named by *$control basis*. The set itself is chosen with *set=* in `$molecule`; without it, the default set of the file is used. Without `basis=`, the basis data are read from the input file itself.

Give the basis file as a **full path**:

```
$control task=optimize basis=/home/user/priroda/basis/basis.in $end
```

A relative name is looked up relative to the directory the program runs in. One user's parallel run stopped after the node list with `application called MPI_Abort(MPI_COMM_WORLD, 1) - process 0` and worked once the basis file was given with its full path.

- Put the name in double quotes if the path contains special characters: `basis="/data/basis sets/basis.in"`. Two single quotes do not work; `basis=''3z.bas''` fails with `error opening file ''3z.bas''`.
- Windows paths work: `basis=D:\NMR\Nature\basis\6-31+G(d)`.
- The relativistic file `basis4.in` needs `four=1` in `$control`. Without it the basis input fails.

Which basis files exist and which sets they contain is described with *$control basis*.

CHAPTER

THREE

GETTING STARTED

This chapter takes you through three things: a single-point energy of water, a geometry optimization of ammonia followed by a Hessian that proves the structure is a minimum, and the parts of the output you need to read along the way. It assumes that Priroda is installed and that you can start it from a shell. *Installing and running* covers that.

Warning

The example inputs in this chapter have not been run yet. They are assembled from inputs published by the developer and by experienced users, adapted to the current basis-set file `basis.in`. They follow the documented syntax, but nobody has checked them against a current Priroda binary. The output excerpts, by contrast, are copied verbatim from real outputs posted by users or shipped with the program. They come from various molecules and versions, so they will not match your runs number for number.

3.1 Before you start

Four practical points save most first-time failures.

- **Plain text with Unix line endings.** Write inputs as plain ASCII with LF line endings and straight quotes. A file saved with Windows (CRLF) line endings gives errors such as `molecule input: ''` or `?rtesian`, or makes `task=scan` silently run an optimization. Convert such files with `dos2unix`.
- **Give the basis file by its full path.** The *basis* keyword names the basis-set file. Relative names have failed on Linux, and an absolute path is the safe choice. The examples below use `/home/user/priroda/basis.in`; replace it with the location of `basis.in` on your machine.
- **Atomic numbers are integers.** Each atom line in `$molecule` starts with the atomic number written as an integer (`8`, not `8.0` and not `O`).
- **Running.** A serial run is `p input output`; a parallel run on one node is `p -np 4 input output`. Use full paths here too.

3.2 A single-point energy

3.2.1 The input

Save the following as `water.in`:

```
$system mem=128 disk=1 $end
$control
```

(continues on next page)

(continued from previous page)

```
 task=energy
 theory=dft
 basis=/home/user/priroda/basis.in
 print=+charges+bonds
$end
$dft functional=PBE $end
$molecule
 charge=0 mult=1
 z-matrix
 set=L1
   8
   1    1    0.96252725
   1    1    0.96252725    2    102.943872
$end
```

Group by group:

- *$system* sets the memory limit in MB and the scratch space. Here `disk=1` means 1 GB of disk for temporary files; a negative value would mean a RAM disk of that many MB. These values are per process.
- *$control* sets the job type and the method. *task=energy* asks for the energy at the given geometry. *theory=dft* is GGA DFT with density fitting, the program's standard method. `print=+charges+bonds` adds Hirshfeld charges and Mulliken populations to the output (see *Charges and bond orders* below).
- *$dft* selects the *functional*. PBE is already the default, so this line only makes the choice visible.
- *$molecule* holds the *charge*, the *multiplicity* (2S+1), the geometry as a Z-matrix (distances in Å, angles in degrees), and the basis set. *set=L1* picks the set named `L1` from `basis.in`. `L1` is also the default set of that file, and users compare it to cc-pVDZ in size.

Note

Hybrid functionals such as `PBE1` or `B3LYP` need *theory=ridft*. With `theory=dft`, older versions silently drop the exact exchange, and newer ones abort. See *$control theory*.

Run it:

```
p water.in water.out
```

3.2.2 What to look for in the output

Near the top, a summary box repeats what the program understood from your input. It is the quickest check that the charge, multiplicity, method and functional are what you meant. This box is from the C60 example on the official site:

```
 +--------------------------------------
 | Number of atoms                    60
 | Charge of molecule                  0
 | Spin multiplicity                   1
 | Number of alpha electrons         180
 | Number of beta  electrons         180
 | Number of core  electrons          60
```

(continues on next page)

(continued from previous page)

```
 | Theoretical Method                 DFT
 | Approximation to E(xc)             PBE
 | Dimension of wavefunction basis    840
 | Dimension of   auxiliary  basis   2400
 | Hamiltonian: non-relativistic
 +-------------------------------------
```

Next comes the SCF iteration table (columns for the energy, the energy change and the gradient). A successful SCF ends with the line `converged!`. If you see `no convergence!` or `SCF is far from convergence`, the energy is not usable; *Solving common problems* lists the fixes.

The total energy follows, in hartree, on the line starting with `E =`, together with its components. This excerpt is from a user's output:

```
 E  =    -232.0211375591
 T  =     230.89636282  V  =    -944.03076786  J  =     312.67315771
 Ex =     -33.07586264  Ec =      -1.37265420
```

`E` is the number you want. The components `T`, `V`, `J`, `Ex` and `Ec` do not add up to `E`, because the nuclear repulsion energy is not printed on these lines. This is expected and is not an error.

Runs that compute a gradient also print the result as a restart block on lines prefixed with `eng>`, which starts with `eng> E=`. Whether a pure `task=energy` run prints this block as well is not documented.

3.3 Optimizing a geometry and confirming a minimum

The standard beginner procedure, as taught on the KNC wiki, alternates between optimizations and Hessians:

1. Optimize the geometry.
2. Check that the optimization converged. If it did not, restart from the last geometry.
3. Take the final geometry and compute a Hessian (force constants and vibrational frequencies) there.
4. If any frequency is imaginary, the structure is not a minimum: re-optimize with that Hessian in the input, then compute a new Hessian.
5. When all frequencies are real, read the thermochemistry.

The Hessian check matters most when the start geometry is symmetric. An experienced user reports that Priroda's optimizer rarely leaves imaginary frequencies, except when the starting structure is symmetric and the optimizer keeps that symmetry; then the check is compulsory.

3.3.1 Step 1: optimize

Save the following as `nh3.in`:

```
$system mem=128 disk=1 $end
$control
 task=optimize
 theory=dft
 basis=/home/user/priroda/basis.in
$end
$optimize tol=1e-5 steps=100 $end
$molecule
 charge=0 mult=1
```

(continues on next page)

(continued from previous page)

```
 z-matrix
 set=L1
   7
   1    1    1.02672223
   1    1    1.02672140    2    104.150240
   1    1    1.02671372    2    104.144601   -3    104.150290
$end
```

The third hydrogen is placed by one distance and two angles: the negative index `-3` means "the angle to atom 3" instead of a dihedral.

tol is the convergence threshold on the gradient. The displacement threshold is then set to 100 times that value. The default is `1e-4`; the tighter `1e-5` used in most published examples is a better starting point for a Hessian. *steps* caps the number of cycles (default 50). If an optimization needs far more than 50 steps, raising the limit rarely helps; supplying a Hessian usually does.

Run it with `p nh3.in nh3.out`.

3.3.2 Step 2: check convergence

Each optimization cycle starts with a line such as `Geometry Optimization Step 0:`, and ends with the maximum gradient (`G(max) = ...`) and displacement (`Max. Displacement = ...`). When both are below the thresholds, the program prints `OPTIMIZATION CONVERGED in N steps!`. Check for it:

```
grep "OPTIMIZATION CONVERGED" nh3.out
```

Right after the marker, the program prints the optimized geometry as a ready-made `$molecule` group, each line prefixed with `MOL>`, followed by the final energy. This excerpt is from the KNC wiki (a relativistic $NpO2^{2+}$ run):

```
 Max. Displacement = 0.0000040

 OPTIMIZATION CONVERGED in 4 steps!
MOL>$molecule
MOL> charge=2 mult=2
MOL> cartesian
MOL> set=L2
MOL>  93     0.00000003  -0.00000001  -0.00000007
MOL>   8    -0.00000002   0.00000001   1.71247831
MOL>   8    -0.00000002   0.00000001  -1.71247823
MOL>$end
MOL> Energy = -28944.57077327
```

If the input geometry was a Z-matrix, as in `nh3.in`, the `MOL>` block reproduces the Z-matrix with the optimized values.

3.3.3 Step 3: take the final geometry

Strip the `MOL>` prefix to get a group you can paste into the next input. The following commands copy `nh3.in` to `nh3_freq.in`, switch the task to `hessian`, drop the old `$molecule` group and everything after it, and append the optimized geometry:

```
sed -e 's/task=optimize/task=hessian/' -e '/\$molecule/,$d' nh3.in > nh3_freq.in
sed -n 's/^MOL>//p' nh3.out >> nh3_freq.in
```

The first command assumes that `$molecule` is the last group of the input, as in all examples here. The `Energy = ...` line that ends up after `$end` does no harm: text outside a group is ignored. A normal optimization prints the `MOL>` block only once. A scan or an IRC prints one per point, so there you would pick the block you need.

Note

If the optimization did not converge, there are no `MOL>` lines. Do not restart from the original geometry: start from the last one reached. The KNC wiki did this with a local script, `lastgeo`, whose code was never published. The limor1 scripts read the current geometry from lines prefixed with lowercase `mol>`, so this keeps the last such block:

```
sed -n 's/^mol>//p' nh3.out | awk '/\$molecule/{buf=""} {buf=buf $0 "\n"} END{printf "
↪%s", buf}'
```

Whether current versions print `mol>` lines at every step is not confirmed. The official `gm` script (below) falls back to the input echo (lines starting with `<$molecule`) when there are no `MOL>` lines.

The official helper scripts `ge` and `gm`

The download site ships two small bash scripts for exactly this loop. Both take a job name (with or without the `.in` or `.out` extension) and work on `name.in` and `name.out` in place:

- `gm nh3` replaces the `$molecule` group in `nh3.in` with the optimized one from `nh3.out` and keeps the old input as `nh3.bak`. Like the commands above, it requires `$molecule` to be the last group, because it replaces everything from `$molecule` onwards. It also appends the final `Energy =` line and, when non-zero, the dipole line.
- `ge nh3` appends the `eng>` lines of `nh3.out` (the `$Energy` group: energy, gradient and, after a Hessian run, the Hessian) to the end of `nh3.in`.

So a cycle with these scripts is: run, `gm`, edit the task, run again. Note that `ge` copies every `eng>` line in the output, so use it on a Hessian output, not on an optimization output that has one block per step.

3.3.4 Step 4: compute the Hessian

After step 3, `nh3_freq.in` looks like this, with your optimized values in place of these coordinates. Two optional lines are added here: `print=+molden` (for viewing the vibrations, see *Viewing results in Molden*) and a `$thermo` group:

```
$system mem=128 disk=1 $end
$control
 task=hessian
 theory=dft
 basis=/home/user/priroda/basis.in
 print=+molden
$end
$optimize tol=1e-5 steps=100 $end
$thermo sigma=3 $end
$molecule
 charge=0 mult=1
 z-matrix
 set=L1
   7
   1    1    1.02672223
```

(continues on next page)

(continued from previous page)

```
   1    1    1.02672140    2    104.150240
   1    1    1.02671372    2    104.144601   -3    104.150290
$end
```

The `$optimize` group is ignored by a Hessian run but does no harm, and it is useful again if you have to go back to step 1. *sigma* is the rotational symmetry number used in the thermochemistry; for ammonia (point group C3v) it is 3. Whether Priroda can work it out itself is not documented, so set it. Temperatures go in *temperature*.

For DFT the Hessian is analytic; the output announces it with `ANALYTIC SECOND DERIVATIVES OF SCF ENERGY`.

3.3.5 Step 5: check the frequencies

The KNC wiki tells you to search for `Normal Mode`: the vibrational frequencies follow that line. For a minimum, **all of them must be real (positive)**. A single imaginary frequency marks a transition state; several mean a higher-order saddle point.

```
grep -A 20 "Normal Mode" nh3_freq.out
```

No verbatim copy of this table has been found in the sources, so it is described here in words. In Priroda 4 and later it is a table with |-separated columns headed `Mode`, `Freq.`, `Mass.` and `IR Int.` (mode number, frequency in cm^{-1}, reduced mass, IR intensity), according to the parsing code of the limor1 scripts. Those scripts recognize imaginary frequencies by an `i` attached to the value. Check how your version prints them before relying on either.

3.3.6 Step 6: if a frequency is imaginary, re-optimize

Put the Hessian you just computed into the optimization input. It is printed as a `$Energy` group on `eng>` lines. Starting from a good Hessian, the optimizer can move off the saddle point; one extra cycle is usually enough.

```
sed -e 's/task=hessian/task=optimize/' nh3_freq.in > nh3_opt2.in
sed -n 's/^eng>//p' nh3_freq.out | awk '/\$Energy/{buf=""} {buf=buf $0 "\n"} END{printf "
↪%s", buf}' >> nh3_opt2.in
```

The second command appends only the last `$Energy` group, after `$molecule`. Do not leave an older `$Energy` group in the file. Then run `nh3_opt2.in` and go back to step 2. (`ge nh3_freq` does the appending in place, if you work with the official scripts.)

3.3.7 Step 7: read the thermochemistry

Once all frequencies are real, the end of the Hessian output holds the zero-point energy and a table of thermodynamic functions. The KNC wiki points to the string `ZPVE`. This is a user's verbatim output (molecule not stated):

```
 E =  -196.328936    E0 =  -196.191669

 ZPVE = 0.137267 a.u. =   86.1366 kcal/mol

 ------------------------------------------------------------------------
 T =  298.000 K      ln(Q)       S          U          H          G
                                cal/mol*K  kcal/mol   kcal/mol   kcal/mol
```

(continues on next page)

(continued from previous page)

```
-------------------------------------------------------------------------
translational      16.9514    38.6542     0.8883     1.4805   -10.0385
rotational         11.1465    25.1313     0.8883     0.8883    -6.6008
vibrational         0.6842     4.7490    87.1466    87.1466    85.7314
total              28.7821    68.5345    88.9232    89.5154    69.0921
-------------------------------------------------------------------------
```

How to read it:

- `E` is the electronic energy in hartree. `E0 = E + ZPVE` is the energy corrected for the zero-point vibrational energy (here $-196.328936 + 0.137267 = -196.191669$).
- The `U`, `H` and `G` columns are **thermal corrections** in kcal/mol, not total values. They already contain the ZPVE. The program does not print the total enthalpy or free energy; you add them yourself.
- Take the corrections from the `total` row and add them to **E, not E0**. Adding them to `E0` would count the zero-point energy twice:

  H = E + ddH, G = E + ddG, with ddH = `H` (total) and ddG = `G` (total).
- Convert units before adding: 1 hartree = 627.51 kcal/mol. For the table above, $G = -196.328936 \times 627.51 + 69.0921 = -123129.28$ kcal/mol, or $-196.328936 + 69.0921 / 627.51 = -196.218831$ hartree.
- The entropy `S` (cal/(mol·K)) is taken from the `total` row as it stands.

Warning

For a **single atom**, the table is wrong: the rotational row contains `-inf/inf` or a spurious RT term. Use only the translational contribution for atoms. Older versions were also reported to put an extra RT into the enthalpy of monoatomic species.

The temperature is printed in the `T =` line. Outputs posted by users show both `298.000 K` and `298.150 K`, so check which value your version uses and set *temperature* explicitly if it matters.

3.3.8 The one-run alternative: `task=optimize+hessian`

You can let one job optimize the structure and then compute the Hessian at the converged geometry. In `nh3.in`, change the task line:

```
$control
 task=optimize+hessian
 theory=dft
 basis=/home/user/priroda/basis.in
 print=+molden
$end
```

This is the only combined task that works: two `task=` lines do not chain jobs, and only the first one runs. `optimize+hessian` existed in Priroda 2, was missing in versions 6 to 8, and is available again from version 9. If the resulting Hessian shows an imaginary frequency, you still need step 6.

3.4 Reading the output

3.4.1 Where things are

The output is plain text, and the results meant for reuse are on lines tagged with a prefix, so `grep` finds them.

Look for	What it is
the summary box (`Number of atoms`, `Theoretical Method`, ...)	what the program understood from the input
`converged!` / `no convergence!`	the end of each SCF
`E =`	total energy in hartree, followed by its components (without nuclear repulsion)
`eng>`	the `$Energy` group: energy `E=`, dipole `D=`, gradient `G=`, and the Hessian after `task=hessian`
`OPTIMIZATION CONVERGED in N steps!`	a converged optimization
`MOL>`	the optimized geometry as a `$molecule` group, then `Energy =`
`Normal Mode`	the start of the frequency table
`ZPVE, T = ... K`	zero-point energy and the thermochemistry table
`Overlap populations:`	Mulliken populations (`print=+bonds`)
`Atomic partitions of the density:`	Hirshfeld charges (`print=+charges`)
`freq>`	vibrations in Molden format (`print=+molden`, Hessian runs)
`mos>`	orbitals in Molden format (`print=+vectors+molden`)

3.4.2 Charges and bond orders

With `print=+charges`, DFT runs print Hirshfeld charges in a block headed `Atomic partitions of the density:`. With `print=+bonds`, SCF runs print Mulliken populations and bond orders in a block headed `Overlap populations:`. The flags can be combined, as in `water.in`. The column layout of these blocks is not shown in the sources; for open-shell molecules the same tables are thought to carry spin populations.

3.4.3 Viewing results in Molden

The KNC wiki treats Molden as the reference viewer. Open files with the `READ` button in the `Miscellaneous` group.

Vibrations. Run the Hessian with *print=+molden* and cut out the `freq>` lines:

```
sed -n 's/^freq>//p' nh3_freq.out > nh3.freq
```

Open `nh3.freq` in Molden to animate the normal modes.

Orbitals. Add `print=+vectors+molden` to `$control`. The orbitals then appear on `mos>` lines:

```
grep '^mos>' water.out | sed 's/^mos>//' > water.mos
```

The `+molden` part is essential: without it, there are no `mos>` lines. The files are large, and Molden can be slow with them.

Geometry. If the optimized geometry is Cartesian, this one-liner turns the `MOL>` block into an XYZ file that Molden reads (the symbol list covers H to Ar; extend it for heavier elements):

```
sed -n 's/^MOL>//p' job.out | awk 'BEGIN{split("H He Li Be B C N O F Ne Na Mg Al Si P S␣
↪Cl Ar",s," ")}
  NF==4 && $1+0>0 {n++; l[n]=s[$1]" "$2" "$3" "$4}
  END{print n; print "optimized geometry"; for(i=1;i<=n;i++) print l[i]}' > job.xyz
```

For a Z-matrix geometry such as `nh3.in`, the `freq>` file from the Hessian run is the easier way to look at the structure.

pri2mol. The Perl script `pri2mol` by the forum user amge, available from `http://limor1.nioch.nsc.ru/quant/program/pri2mol/`, converts a Priroda output into Molden and XYZ files: geometries, vibrations, orbitals, NMR shifts and density critical points. Priroda's own viewer, `xm`, is on the download site.

3.5 Where next

- *Input syntax* explains groups, keyword abbreviation, arrays and restart blocks.
- *Installing and running* covers memory, scratch disks and parallel runs.
- The *input reference* describes every known keyword, group by group.
- *Solving common problems* helps when a run fails.

CHAPTER

FOUR

INPUT SYNTAX

`input.txt` describes the input as "a kind of NAMELIST free format". It is close to the GAMESS-US format: keywords are grouped, and each group starts with `$name` and ends with `$end`. This chapter covers the rules that apply to all groups. The individual keywords are in the *input reference*.

A complete input for an NH3 optimization, adapted from the KNC wiki FAQ:

```
$system mem=512 disk=-512 path=/tmp $end
$control
 task=optimize
 theory=dft
 basis=/home/user/priroda/basis.in
$end
$optimize tol=1e-5 steps=100 $end
$molecule
 charge=0 mult=1
 z-matrix
 set=L1
   7
   1    1    1.02672223
   1    1    1.02672140    2    104.150240
   1    1    1.02671372    2    104.144601   -3    104.150290
$end
```

4.1 Groups

A group starts with `$` followed by the group name and ends with `$end`. It may sit on one line or span several:

```
$scf conv=1e-7 $end

$scf
 convergence=1e-7
 procedure=bfgs
$end
```

Inside a group, `keyword=value` pairs are separated by spaces or line breaks, and several pairs can share a line (`theory=qm_n3 basis=qm.in`). Blank lines inside a group are allowed.

The order of the groups does not matter, and neither does the order of keywords within a group. The exceptions are `$molecule` and `$Energy`, whose contents are ordered data.

Groups you do not need can be left out; most keywords have sensible defaults. `input.txt` advises changing only what the problem requires.

Note

Every input in the sources starts each group on a new line. Whether two groups can share a line (`$scf conv=1e-7 $end $grid acc=1e-9 $end`) is not documented. Start each group on its own line.

4.2 Keywords

4.2.1 Case

Group names, keyword names and keyword values are case-insensitive: `$System` and `$system`, `theory=riDFT` and `theory=ridft`, `functional=pbe` and `functional=PBE` are the same. File names are the exception, because Linux file names are case-sensitive. This applies to `basis=`, `save=` and `path=`.

4.2.2 Abbreviations

Within a group, a keyword can be shortened to a prefix. The example in `input.txt` is `mem` for `memory`. Inputs in the sources use, among others:

Short form	Keyword
`mem=`	*$system memory*
`conv=`, `iter=`, `proc=`	*$scf convergence*, *iterations*, *procedure*
`func=`	*$dft functional*
`acc=`, `accur=`	*$grid accuracy*
`tol=`, `cart=`, `val=`	*$optimize tolerance*, *cartesian*, *value*
`disp=`	*$d2edr2 displace*
`nuc=`	*$control nucleus*
`rout=`	*$integral routine*

Keywords belong to their group. The same name can mean different things in different groups: `steps` in *$optimize* is the maximum number of optimization cycles, while `steps` in *$d2edr2* selects displacement steps of a numerical Hessian.

Warning

A prefix must stay unambiguous within its group. How Priroda resolves a prefix that matches two keywords is not documented. In version 6, when `$optimize` contained both `step=` and `steps=`, the maximum number of steps was taken from `step`. In inputs you keep or share, prefer the full keyword names; they are also easier to search for.

A misspelled or unknown keyword name stops the run with a message naming the group, of the form `*** error in '$scf' : unknown name erations`. That particular message comes from a parser problem with `convergence` and `iterations` in the same `$scf` group; see *$scf convergence*.

4.3 Values

Scalars

Integers (`steps=100`), reals in plain or exponent notation (`trust=0.2`, `tol=1e-5`) and words (`task=optimize`). A sign is allowed: `charge=+1`.

Lists
: Values separated by commas, for keywords that take several numbers: `tol=1e-5,1e-4`, `mem=2048, 256`, `temperature=203, 223, 248, 298`. Spaces after the commas are allowed, and some lists are also written with spaces between groups of values (`fix=1,1,2,0,0 3,2,1,4,3`). If you give fewer values than the keyword holds, the rest keep their defaults: `$scf conv=1e-10` sets only the SCF threshold and leaves the CP-SCF threshold at `1e-3`.

Indexed arrays
: `name(i)=a,b,...` assigns values to an array starting at element i and leaves the other elements unchanged. `$guess na(85)=86,85 $end` sets element 85 of `na` to 86 and element 86 to 85, which swaps two orbitals. The same form is used for *$optimize value* (`val(5)=2.0,2.1`), *$nmr standard* (`standard(6)=181.9921`) and *$atoms core*.

Flag strings
: *$control print* takes a string of options, each switched on with `+` or off with `-`: `print=+vectors+molden`, `print=-internals+molden`. The leading `+` may be omitted (`print=charges+bonds`).

Quoted strings
: A file name or path may be put in double quotes, and must be if it contains special characters: `basis="3z.bas"`, `path="/home/user/priroda/scratch"`. Use the straight ASCII double quote `"`. Typographic quotes break the name, and two single quotes do not count as a double quote: `basis=''3z.bas''` fails with `error opening file ''3z.bas''`.

4.4 The `$molecule` group

$molecule is read as ordered data rather than as keyword pairs:

1. `charge=` and `mult=` (spin multiplicity 2S+1), both optional, with defaults 0 and 1;
2. one of the words `cartesian` or `z-matrix`;
3. one line per atom, optionally preceded by `set=` lines that choose the basis set for the atoms that follow.

The first items may share the opening line:

```
$molecule charge=0 mult=1 cartesian
 set=L1
  8    0.000000    0.000000    0.117300
  1    0.000000    0.757200   -0.469200
  1    0.000000   -0.757200   -0.469200
$end
```

Each atom line begins with the atomic number written as an integer. Cartesian coordinates are in Å unless `unit=b` is given. The Z-matrix forms, mixed Cartesian and Z-matrix input, isotope masses and per-atom basis sets are described in the *$molecule* reference.

4.4.1 Several `$molecule` blocks

Some tasks read more than one `$molecule` group from one input:

- `task=transit`, `task=saddle` and `task=path` take two blocks, the start and the end structure. The atoms must be numbered the same way in both.
- `task=uniq` takes any number of blocks and filters out duplicate structures; the official example has 157.

These tasks are undocumented; see *$control task* and *$uniq*.

4.5 Text outside groups

Everything outside a `$name ... $end` group is ignored. There is no documented comment character, but this rule serves the same purpose, and the official example inputs use it to switch input off:

- **Delete the `$`** to disable a whole group. `integral dir=1 $end` in the official 2023–2025 example inputs is plain text, not an `$integral` group.
- **Move keywords after `$end`** to disable them. In the example below, `v=` and `h=` are outside `$dynamic` and have no effect.
- **Keep notes after `$end`.** Helper scripts append result lines such as `Energy = ...` after the `$molecule` group, where they do no harm.

```
integral dir=1 $end
$dynamic
 e=0.001
 d=0.03
 n=1000,100
$end
 v=500.0
 h=0.1
$molecule
 ...
$end
 Energy =   -783.83738426
```

Note

That disabled groups and keywords are ignored is inferred from the official example files, which use this pattern throughout. It has not been tested what the program does with the stray `$end` that a disabled group leaves behind. Text inside a group is parsed as keywords, so do not put notes there.

One user's input opened a group with `&` instead of `$` (`&dft func=PBE $end`) and ran. That group set only the default functional, so the run does not show whether `&` is accepted. Always use `$`.

4.6 Appending results of earlier runs

Priroda writes results that can be reused as input into the main output file. Each such line carries a prefix, for example `eng>` or `num>`. Strip the prefix and append the lines to the next input. The program has no separate restart file for these data; SCF orbitals are the exception, saved with *$control save*.

4.6.1 `$Energy` from `eng>` lines

A run prints its energy, dipole, gradient and, after `task=hessian`, the Hessian as an *$Energy* group on lines prefixed with `eng>`:

```
eng>$Energy
eng> E=-2.894457077327e+04
eng> D= 0.00000000e+00  0.00000000e+00  0.00000000e+00
eng> G= 3.53877831e-09  1.65504544e-08 -3.64771621e-07
eng>   -1.31128901e-07 -6.73350427e-09 -2.19780956e-06
```

(continues on next page)

(continued from previous page)

```
eng>    1.27590122e-07 -9.81695012e-09  2.56258122e-06
eng>$end
```

To reuse it, remove the prefix and append the block to the end of the next input, after `$molecule`:

```
sed -n 's/^eng>//p' mol.out >> mol.in
```

The `ge` script on the official site does the same. It also skips the first block when the output contains a `NUMERICAL AND ANALYTICAL GRADIENT` section.

Warning

An optimization prints an `eng>` block at every step, and the command above appends all of them. Append only the block you need, normally the one from a `task=hessian` run. It is not documented what the program does with several `$Energy` groups in one input.

A stored Hessian is required by *$optimize saddle=1* and `task=irc`. It also helps difficult minimizations, and it lets you redo the vibrational analysis or the thermochemistry without recomputing.

4.6.2 `num>` lines

Partial results of a numerical calculation are printed on lines prefixed with `num>`. There are two uses:

- A numerical Hessian split over several runs with *$d2edr2 steps*. Collect the `num>` lines of all partial outputs into one input, add `$d2edr2 steps=-1 $end`, and run once more:

  ```
  grep "num>" mol.out{1,2,3,4} | sed "s/\.*num>//g" >> mol.in
  ```

  The `sed` expression also removes the file-name prefix that `grep` adds for several files.

- Raman intensities. A `task=dipole` run writes `num>` lines; append them to the input of a `task=hessian` run.

4.6.3 Optimized geometry from `MOL>` lines

After `OPTIMIZATION CONVERGED`, the final geometry is printed as a ready-made `$molecule` group on lines prefixed with `MOL>`. Strip the prefix to continue from that geometry. The `gm` script on the official site replaces the `$molecule` group of the input with it. Because `gm` replaces everything from `$molecule` to the end of the file, keep `$molecule` as the last group of your inputs, followed only by appended result blocks.

4.7 Common mistakes

Non-integer atomic numbers
: `25.0` or `8.0` instead of `25` or `8` in `$molecule` stops the run after `molecule input:  ''`, with `illegal charge/multiplicity`. Element symbols (`O`) are not accepted either.

Typographic quotes
: Word processors and some web pages turn `"` into curly or angle quotes, which break file names in `basis=` and `path=`. Retype them as straight quotes.

Two `task=` lines
: A second `task=` line does not start a second job; only the first task runs. `task=optimize+hessian` is the only combination the program understands. Run other sequences as separate jobs.

Windows line endings (CRLF)
: The run stops in `$molecule` with `molecule input:  ''` followed by a garbled `' ?matrix` or `?rtesian`. Convert the file with `dos2unix`; see *Installing and running*.

Hybrid functional without `theory=ridft`
: Syntactically valid but wrong. See *$control theory*.

`basis4.in` **without** `four=1`
: The relativistic basis file needs `four=1` in `$control`.

More input problems, with their exact error messages, are listed in *The input is not read, or is read wrongly*.

CHAPTER

FIVE

INPUT REFERENCE

Group	Purpose
$system	Memory, disk and scratch-directory limits.
$control	Job type, level of theory, basis-set file, Hamiltonian and printing.
$molecule	Charge, multiplicity, geometry (Cartesian and/or Z-matrix) and per-atom basis sets.
$guess	Initial guess for the SCF vectors.
$scf	Spin restriction, convergence, iteration limits and SCF solver.
$dft	Exchange–correlation functional.
$grid	Adaptive integration grid for DFT.
$integral	Integral evaluation for RI methods.
$atoms	Per-element core definitions for correlated (HF-based) methods.
$optimize	Geometry optimization, saddle-point search, IRC and relaxed scans.
$d2edr2	Numerical Hessian by finite differences of the gradient.
$thermo	Thermochemistry after a Hessian calculation.
$nmr	Reference shieldings and extra points for NMR shielding calculations.
$Energy	Results of an earlier run (energy, gradient, Hessian, dipole data), read back as input.
$dynamic	Molecular dynamics with a thermostat.
$uniq	Duplicate-structure filtering over several `$molecule` blocks.

5.1 $system

Memory, disk and scratch-directory limits.

All limits apply **per MPI process**. For example, `disk=10` on four processes can use 40 GB. In a parallel run, most of the memory and all of the disk storage are used by the master process (rank 0).

5.1.1 memory

Type: list
Default: *none stated*
Units: MB
Status: current · not verified
Sources: input.txt; 01 §4.1 (two values); 06 §3 (estimate); 03 §4, 04 #196 (per process)

Memory limit in megabytes. A second value sets the limit for the other MPI ranks: `mem=2048,256` gives rank 0 2048 MB and every other rank 256 MB. A single value applies to all ranks. The output echoes the result as `Memory: <mem>m + <|disk|>m (<mem2>m + <|disk2|>m)`.

Note

For DFT, the v5 documentation estimates the memory as about $8 \cdot N^2 \cdot 3$ bytes for energy and gradient, and twice that for an analytic Hessian, where N is the basis dimension. `Dft.rtf` gives an old default of 16 MB.

5.1.2 `disk`

Type: list
Default: *none stated*
Status: current · not verified
Sources: input.txt; Dft.rtf (one file per GB); 01 §4.1; 04 §0, #468; 03 §17

Positive values set the disk space in gigabytes; one scratch file is opened per gigabyte, which sidesteps per-file size limits. Negative values set the amount of memory, in megabytes, used to emulate disk (a RAM disk). A second value applies to the other MPI ranks, as for `memory`; the official example inputs typically give them `-1`.

Note

With a RAM disk, `memory` plus —`disk`— (times the number of processes) must fit in physical RAM, or the run fails with `malloc(...) failed`. Example for a 24 GB node: `memory=9000 disk=-14000`. Keeping scratch in RAM gave a 6× speed-up for RI-MP2 on one cluster.

Example:

```
$system mem=2048,256 disk=-256,-1 $end
```

5.1.3 `path`

Type: string
Default: *none stated*
Status: current · not verified
Sources: input.txt; 03 §2; 04 #465

Default: the `TMPDIR` environment variable, otherwise the current working directory.

Directory for temporary disk files. For parallel runs you can give a colon-separated list with one path per process: `path=/tmpa:/tmpb:/tmpc:/tmpd` makes process 1 write to `/tmpa`, process 2 to `/tmpb`, and so on. Those directories should be on different physical drives. The value may be double-quoted.

Note

For all methods except GGA DFT, performance degrades "terribly" if more than one process reads and writes large files on the same physical drive. Never put temporary files on NFS. On Windows (Cygwin), which has no `/tmp`, use `path=.`.

Example:

```
$system path=/tmpa:/tmpb:/tmpc:/tmpd $end
```

5.2 $control

Job type, level of theory, basis-set file, Hamiltonian and printing.

5.2.1 task

Type: enum
Default: `gradient`
Status: current · not verified
Sources: input.txt; Dft.rtf (default gradient, check); 02 §3.2 (pri: default gradient); 03 §5; 04 #69-#73, #145, #478, #497; 01 §4.2

Job type.

Value	Meaning
`energy`	Energy at the given geometry.
`gradient`	Energy and nuclear gradient. Costs little more than the energy. (**default**)
`hessian`	Second derivatives (force constants) followed by vibrational analysis and thermochemistry.
`optimize`	Geometry optimization, including saddle points with `$optimize saddle=1`.
`optimize+hessian`	Optimization followed by a Hessian at the final geometry. The only combined task that works. Present in v2 and v9 and later, absent in v6.
`irc`	Intrinsic reaction coordinate. Needs a Hessian at the TS in the input (`$Energy`).
`scan`	Relaxed potential-energy-surface scan over the coordinates in `$optimize fix`.
`nmr`	NMR shielding tensors. Works only with `theory=dft`.
`dipole`	Numerical dipole polarizabilities and their derivatives with respect to nuclear coordinates (used for Raman intensities).
`check`	Process the geometry only. *(undocumented)*
`extremal`	Gradient-extremal path (v9 banner "Calculation of Gradient Extremal Path"); slow. *(undocumented)*
`transit`	Synchronous transit in internal coordinates. Needs two `$molecule` blocks (start and end). Reported working in v8. *(undocumented)*
`saddle`	Approximate saddle-point search from two `$molecule` blocks (start and end). *(undocumented)*
`path`	Minimum-energy-path optimization from two `$molecule` blocks (start and end). *(undocumented)*
`conical`	Conical intersection of states 0 and 1 (v9 banner). Untested. *(undocumented)*
`anharmonic`	Anharmonic corrections to vibrational energies; slow. *(undocumented)*
`opt_n1`	Optimizer for very large systems (used for 8060 atoms with `theory=qm_n2`). Pairs with `$optimize mod=1`. *(undocumented)*
`thermostat`	Molecular dynamics with a thermostat; see `$dynamic`. *(undocumented)*
`uniq`	Filter duplicate structures among several `$molecule` blocks; see `$uniq`. *(undocumented)*

Note

Two `task=` lines do not chain jobs: only the first one runs. Apart from `optimize+hessian`, sequences such as optimize→NMR need separate runs or an external driver script. `task=critical` does not work;

density critical points come from `print=+density`. `task=QM` appears only in the header of the shipped `qm.in` parameter file and is not a user task. The binary contains further task words (`IRC4`, `PATHX`, `PATHR`, `STERIC`, `POLARIZABILITY`, `MMDYNAMIC`, and others) that are untested or need data that is not distributed.

See also: *saddle ($optimize)*, *points ($optimize)*, *print ($control)*

5.2.2 `theory`

Type: enum
Default: `dft`
Status: current · not verified
Sources: input.txt; Dft.rtf (default dft); 06 §3 (v5 ab initio list); 03 §5 (Evgeniy 2011 test); 04 #96, #99, #307-#310; 01 §3.1, §4.2

Theoretical method.

Value	Meaning
`dft`	GGA DFT with density fitting. Hybrid functionals are **not** applied here: see notes. (**default**)
`ridft`	Hybrid DFT with exact exchange via the resolution-of-identity (RI) approximation. Required for hybrid functionals.
`DFT_n4`	Newer DFT variant, used in the official example inputs with `func=LPBEVV` (2025). *(undocumented)*
`tddft`	GGA time-dependent DFT (no hybrid TDDFT).
`rimp2`	RI-MP2.
`hf`	Hartree–Fock. *(undocumented)*
`rihf`	RI Hartree–Fock. *(undocumented)*
`cis`	CIS (RHF/UHF). Slow and disk-hungry. *(undocumented)*
`mp2`	Conventional MP2 (RHF, UHF, ROHF). Very disk-hungry. *(undocumented)*
`mp3`	MP3. *(undocumented)*
`mp4dq`	MP4(DQ). *(undocumented)*
`mp4sdq`	MP4(SDQ). *(undocumented)*
`mp4sdtq`	MP4(SDTQ) (RHF). *(undocumented)*
`ccs`	CCS. *(undocumented)*
`ccd`	CCD. *(undocumented)*
`ccsd`	CCSD (RHF/UHF).
`ccsd4t`	CCSD(T).
`ccsdsd`	Linear-response (EOM) CCSD for excited states (RHF). *(undocumented)*
`qm_n3`	Laikov's semiempirical QM model. Needs `basis=qm.in`; parameters exist for H, Li–F, Na–Cl, Sc–Cu. *(undocumented)*
`qm_n2`	Earlier variant of the QM model, used for very large systems with `task=opt_n1`. *(undocumented)*

Note

Hybrid trap. Hybrid functionals (`PBE1`, `B3LYP`, ...) take effect only with `theory=ridft`. With `theory=dft` or `tddft`, older versions silently drop the exact exchange and run a pure GGA. Newer ones abort with `MPI_Abort` after reading the basis twice.

The ab initio list comes from the v6 documentation and a 2011 user test, which was energy-only. These

were reported **not** working in v6: `mp4`, `rimp4`, `rimp4sdtq`, `rimp4sdq`, `riccsd`, `riccsd4t`. They either crash or return the HF energy. The binary contains many more method words (CASSCF, MRCISD, CCSDT, ...); none has been reported working.

See also: *functional ($dft)*

5.2.3 `state`

Type: list
Default: *none stated*
Status: current · not verified
Sources: input.txt

For excited-state methods. The first value is the excited state n whose properties are computed; the second is the number of states N included in the iterative solution of the equations (`state=n,N`).

5.2.4 `basis`

Type: string
Default: *none stated*
Status: current · not verified
Sources: input.txt; Dft.rtf; 01 §4.11, §6; 03 §5; 04 §0

Default: basis-set data are read from the input file itself.

Name of the file containing the basis-set data (and ECPs, or QM parameters for `qm.in`). The name may be double-quoted, which is needed when the path contains special characters. Use straight quotes: typographic quotes break parsing.

Note

Shipped files include `basis.in` (Λ/L family, default set `L1`), `basis4.in` (scalar-relativistic L family; needs `four=1`), `3z.bas` (DFT only, Z ≤ 54), `sbk.bas` (SBK ECPs, up to Z = 86), `b2*.in`, `b2u*.in`, `b4s*.in`, and `qm.in`. An absolute path is safest. Chapter "Basis sets" has the full family list.

See also: *set ($molecule)*, *four ($control)*

5.2.5 `four`

Type: switch
Default: `0`
Status: current · not verified
Sources: input.txt; 01 §3.3; 04 #207-#208, #336

Four-component option.

Value	Meaning
`0`	Non-relativistic theory. (**default**)
`1`	Scalar-relativistic theory (K. G. Dyall, J. Chem. Phys. 100 (1994) 2118–2127). The output reports "Hamiltonian: scalar-relativistic".

Note

Required with the relativistic basis file `basis4.in`. Forgetting it is a common cause of failed runs with that file.

See also: *nucleus ($control)*, *d1small ($scf)*

5.2.6 `nucleus`

Type: enum
Default: *none stated*
Status: current · not verified
Sources: input.txt; 01 §4.2; 06 §1

Default: `finite` for four-component theory, `point` for non-relativistic theory.

Nuclear model. The official example inputs write it as `nuc=finite`.

Value	Meaning
`finite`	Gaussian nuclear model (L. Visscher, K. G. Dyall, At. Data Nucl. Data Tables 67 (1997) 207–224).
`point`	Point-nucleus model.

Note

A basis optimized for a point nucleus must not be used with the finite-nucleus model, and vice versa.

See also: *four ($control)*

5.2.7 `print`

Type: flags
Default: *none stated*
Status: current · not verified
Sources: input.txt; Dft.rtf; 06 §3; 04 #376, #460, #498-#502, #509

Printing options. Flags are appended one after another; `+` switches an option on and `-` switches it off. The leading `+` can be omitted (`print=charges+bonds`).

Value	Meaning
`+charges`	Hirshfeld charges (DFT methods), printed as "Atomic partitions of the density".
`+bonds`	Mulliken populations and bond orders (SCF only), printed as "Overlap populations".
`+esr`	ESR parameters.
`+vectors`	SCF vectors.
`+molden`	MOLDEN data: vibrations on `freq>` lines for `task=hessian`; with `+vectors+molden` also orbitals on `mos>` lines.
`+density`	Critical points of the electron density (QTAIM). Combine with `+charges` to label them. *(undocumented)*
`-internals`	Suppress the printout of internal coordinates. *(undocumented)*

Example:

```
$control print=+vectors+molden $end
```

5.2.8 save

Type: string
Default: *none stated*
Status: current · not verified
Sources: input.txt; Dft.rtf; 04 #306

File name for saving SCF vectors. With `$guess read=1` the vectors are read back from the same file. This is the basis of the SCF restart technique.

See also: *read ($guess)*

5.3 $molecule

Charge, multiplicity, geometry (Cartesian and/or Z-matrix) and per-atom basis sets.

Unlike the other groups, `$molecule` holds **ordered data**. It starts with `charge=` and `mult=`, then one of the keywords `cartesian` or `z-matrix`, followed by one line per atom.

Cartesian lines: `Z x y z`. **Z is the atomic number written as an integer**: `8`, not `8.0` and not `O`. A value such as `25.0` breaks parsing with `molecule input: ''`. The coordinates are in Å unless `unit=b` is given.

Z-matrix lines, `Z j r k a l b`, can combine distances, angles and torsions in four ways:

Form	Meaning
`q_i j r_ij k a_ijk l b_ijkl`	distance, angle, torsion
`q_i j r_ij k a_ijk -l a_ijl`	distance and two angles
`q_i j r_ij -k r_ik l b_ijkl`	two distances and torsion
`q_i j r_ij -k r_ik -l r_il`	three distances

Distances are in Å, and angles and torsions are in degrees. In a pure Z-matrix the first atom gives only its atomic number, the second only a distance, and the third two internal coordinates. The two formats can be mixed: put the Cartesian atoms first, then the remaining atoms as a Z-matrix.

The program does not use or impose point-group symmetry unless the newer `sym=` option is given. Dummy atoms are not supported, so counterpoise (BSSE) calculations cannot be done; NICS probe points go in `$nmr points=` instead. Some tasks (`transit`, `saddle`, `path`, `uniq`) read **several** `$molecule` blocks from one input. Text after `$end`, such as the `Energy = ...` lines that helper scripts append, is ignored.

5.3.1 charge

Type: integer
Default: `0`
Status: current · not verified
Sources: input.txt; Dft.rtf (default)

Molecular charge.

5.3.2 mult

Type: integer
Default: 1
Status: current · not verified
Sources: input.txt; Dft.rtf (default)

Spin multiplicity, 2S+1.

5.3.3 mass

Type: real
Default: *none stated*
Units: amu
Status: current · not verified
Sources: input.txt

Optional atomic mass written after an atom's coordinates, for isotopes.

5.3.4 set

Type: string
Default: *none stated*
Status: current · not verified
Sources: input.txt; 03 §5 (default set); 01 §4.11; 06 §1

Default: the set named by `default=` in the basis-file header (`L1` in `basis.in`).

Basis set for the atoms that follow. The name must match one in the basis-set file. It can appear several times in the group. The orbital basis is paired with its own auxiliary (fitting) basis, so you cannot combine arbitrary orbital and auxiliary sets.

Example:

```
$molecule
 z-matrix
 set=B2
   8
 set=B1
   1    1    0.96252725
   1    1    0.96252725    2    102.943872
$end
```

See also: *basis ($control)*

5.3.5 unit

Type: string
Default: *none stated*
Status: undocumented · not verified
Sources: 01 §4.7

Length unit of the coordinates. The default unit is Å.

Value	Meaning
b	Coordinates in bohr.

5.3.6 sym

Type: list
Default: *none stated*
Status: undocumented · not verified
Sources: 01 §4.7; 04 #481

Used as `sym=1,0.01` in the official 2023–2025 example inputs, apparently to enable symmetry with tolerance 0.01. Newer versions report the detected point group (`symmetry: D6h`).

See also: *sym ($optimize)*

5.4 $guess

Initial guess for the SCF vectors.

5.4.1 read

Type: switch
Default: *none stated*
Status: current · not verified
Sources: input.txt; 04 #306; 05 §1

Source of the starting SCF vectors.

Value	Meaning
1	Read SCF vectors from the file named by `$control save=`.
0	Extended Hückel guess.

Note

Reading saved vectors is the standard way to restart a stalled SCF, and to reuse converged orbitals in a follow-up job such as a Hessian after an optimization.

See also: *save ($control)*, *iterations ($scf)*

5.4.2 na

Type: indexed-array
Default: *none stated*
Status: current · not verified
Sources: input.txt

Default: `na(1)=1,2,3,...,N` (the natural order).

Reordering of the starting vectors for alpha spin.

Example:

```
$guess na(25)=26,25 $end    # swap the 25th and 26th vectors
```

See also: *nb ($guess)*

5.4.3 nb

Type: indexed-array
Default: *none stated*
Status: current · not verified
Sources: input.txt

Default: `nb(1)=1,2,3,...,N`.

Reordering of the starting vectors for beta spin.

See also: *na ($guess)*

5.4.4 mix

Type: switch
Default: `0`
Status: undocumented · not verified
Sources: input.txt (annotation corrupted); 03 #0, §7; 06 §3

Breaks the equivalence of the alpha and beta electrons in the highest occupied orbital. It is meant for open-shell singlets (for example singlet biradicals) and is only sensible with `$scf restrict=0`.

Value	Meaning
`0`	No mixing. (**default**)
`1`	Mix the HOMO and LUMO to break alpha/beta equivalence.

Note

Users report that with non-RI DFT, `mix=1` can converge back to the restricted solution (same energy as RKS). RI-DFT, or a hybrid, broke the symmetry in the reported cases.

See also: *restrict ($scf)*

5.5 $scf

Spin restriction, convergence, iteration limits and SCF solver.

5.5.1 restrict

Type: switch
Default: *none stated*
Status: current · not verified
Sources: input.txt

Default: `1` if the system has an even number of electrons, otherwise `0`.

Spin restriction.

Value	Meaning
`0`	Spin-unrestricted.
`1`	Spin-restricted (available only for a few methods).

See also: *mix ($guess)*

5.5.2 convergence

Type: list
Default: `1e-6,1e-3`
Status: current · not verified
Sources: input.txt; 01 §4.5 (output echo); 05 §2; 04 #322-#324

Default: the first value is for SCF and the second for CP-SCF.

SCF and CP-SCF convergence thresholds. The defaults are recommended. To save time in difficult cases you may loosen the SCF value to `1e-5` for preliminary geometry optimizations.

Note

Two problems are reported when `convergence` and `iterations` appear together in `$scf`. In one, the group stops being read or fails with `*** error in '$scf' : unknown name erations`. In the other, the pair has no effect unless `procedure=bfgs` is also given. Reports disagree on whether keyword order matters. Until this is verified, check the `scf options:` echo line in the output.

See also: *iterations ($scf)*, *procedure ($scf)*

5.5.3 iterations

Type: list
Default: `16,16`
Status: current · not verified
Sources: input.txt; 01 §4.5; 04 #319; 05 §5; Dft.rtf; 02 §3.2 (pri help); 04 #306

Default: `16,16` is what v6, v13 and v15 echo for DFT. A HF run for CCSD showed `48,16`, so the default may depend on the method. `Dft.rtf` gives 30 (older version).

Maximum number of iterations and of microiterations (`iterations=it,micro`).

Note

Do not raise this limit hoping for convergence: understand why SCF is not converging. A recommended technique for difficult cases is to cap the run at `iterations=10`, save the vectors (`$control save=`) and restart with `$guess read=1`, repeating as needed.

See also: *convergence ($scf)*, *read ($guess)*

5.5.4 procedure

Type: enum
Default: `nr`
Status: current · not verified
Sources: input.txt; Dft.rtf; 04 #320

Default: `nr` for GGA DFT.

Algorithm for solving the SCF equations. If one procedure fails, try the other.

Value	Meaning
`nr`	Newton–Raphson quadratically convergent. Can recover even from a wrong starting electronic configuration but may blow up in hard cases. (**default**)
`bfgs`	BFGS Hessian update. Less prone to blowing up but usually slower and sometimes converges poorly.

Example:

```
$scf convergence=1e-7 procedure=bfgs iterations=250 $end
$grid accuracy=1e-9 $end
```

5.5.5 `d1small`

Type: switch
Default: `0`
Status: current · not verified
Sources: input.txt

Small-component density in quadratically convergent SCF.

Value	Meaning
`1`	Include the small-component density in the linear equations of quadratically convergent SCF (scalar-relativistic pure DFT).
`0`	Discard these contributions. This is faster, but SCF may fail to converge for molecules containing the heaviest atoms. (**default**)

See also: *four ($control)*

5.5.6 `core`

Type: integer
Default: *none stated*
Status: current · not verified
Sources: input.txt

Default: computed from the atom table; see `$atoms core`.

Number of core electrons for HF-based methods.

See also: *core ($atoms)*

5.6 `$dft`

Exchange–correlation functional.

5.6.1 `functional`

Type: enum
Default: `PBE`
Status: current · not verified
Sources: input.txt; 01 §3.2; 03 §5 (strings); 04 §0

Default: PBE (the default GGA); PBE1 for theory=ridft.

Exchange–correlation approximation for DFT methods.

Value	Meaning
PBE	Perdew–Burke–Ernzerhof, Phys. Rev. Lett. 77 (1996) 3865. (**default**)
mPBE	Adamo–Barone modification of PBE, J. Chem. Phys. 116 (2002) 5933.
BLYP	Becke–Lee–Yang–Parr, Phys. Rev. A 38 (1988) 3098; Phys. Rev. B 37 (1988) 785.
OLYP	Handy–Cohen exchange + LYP correlation, J. Chem. Phys. 117 (2002) 1441.
PBE1	PBE hybrid with 1/4 exact exchange, J. Chem. Phys. 110 (1999) 6158. Default for theory=ridft.
B3LYP_PW	Becke three-parameter hybrid with PW local correlation (see notes).
B3LYP	Becke three-parameter hybrid (see notes).
LPBEVV	Newer functional used in the official example inputs with theory=DFT_n4 (2025). *(undocumented)*
LPBEVVV	Newer functional used in the official example inputs with theory=ridft (2023–2025). *(undocumented)*

Note

The B3LYP form in Priroda is Exc = 0.2·Ex(HF) + 0.8·(0.9·Ex(B) + 0.1·Ex(S)) + 0.81·Ec(LYP) + 0.19·Ec(LSD). Ec(LSD) is the local correlation of either Phys. Rev. B 45 (1992) 13244 (PW) or Can. J. Phys. 58 (1980) 1200 (VWN). input.txt points out that some commercial programs use VWN, although Becke himself used PW (J. Chem. Phys. 98 (1993) 5648).

Hybrids (PBE1, B3LYP, B3LYP_PW) require theory=ridft. With theory=dft or tddft they either run silently without exact exchange or abort; see $control theory.

No empirical dispersion correction (D3 or similar) is available. The v6 binary contains further functional names (VWN5, SLYP0, BPBE, HFLSD, HFPBE, B2PBE, B3PBE, HFLYP, B1LYP, B2LYP), none of them documented or tested.

See also: *theory ($control)*

5.7 $grid

Adaptive integration grid for DFT.

5.7.1 accuracy

Type: list
Default: 1e-8,1e-5
Status: current · not verified
Sources: input.txt; 01 §4.4 (output echo); Dft.rtf

Default: 1e-8 for the final grid and 1e-5 for the coarse grid used in the first SCF iterations, as echoed by v13/v15 (grid: accuracy=1.0e-08,1.0e-05). Dft.rtf gives 1e-7 (older version).

Target accuracy of the exchange–correlation energy per atom for the adaptively generated grid (DFT methods). The default, 1e-8, is a fine grid. Use 1e-7 to speed up geometry optimizations with tolerance<=1e-4,

and 1e-9 for very accurate calculations. The grid grows during the SCF; the output reports each size (`Grid: ... points`).

Note

`Dft.rtf` suggests 1e-6 for rough optimizations (`tol=1e-3`) and 1e-8 for floppy molecules optimized to `tol=1e-5`. It says that asking for more than 1e-8 "makes no deep sense", because the integration formulas cannot reach unlimited accuracy. This conflicts with `input.txt`'s 1e-9 for very accurate work.

See also: *tolerance ($optimize)*

5.8 $integral

Integral evaluation for RI methods.

5.8.1 direct

Type: switch
Default: `0`
Status: undocumented · not verified
Sources: input.txt (annotation corrupted); 03 #0, §17; 05 §2

Selects direct (recomputed) Coulomb integrals for RI methods.

Value	Meaning
`0`	Store integrals. (**default**)
`1`	Recompute Coulomb integrals (direct mode) in `theory=rimp2` and `theory=ridft`.

Note

Direct mode roughly halves disk use, but it is slower. It is also the reported workaround when a small RI-MP2 job claims not to fit in memory, or crashes with `write VMNN(...)?`.

Example:

```
$integral direct=1 rout=2 $end
```

5.8.2 routine

Type: integer
Default: `0`
Status: undocumented · not verified
Sources: input.txt (annotation corrupted); 03 #0

Algorithm for the two-electron integrals in correlated methods. `input.txt`'s example uses `rout=2`, but no source explains the value 2.

Value	Meaning
0	Default algorithm. (**default**)
1	Slightly slower but parallelizes better over many processes.

5.9 $atoms

Per-element core definitions for correlated (HF-based) methods.

5.9.1 core

Type: indexed-array
Default: *none stated*
Status: current · not verified
Sources: input.txt

For HF-based methods, the number of core shells for L=0,1,2,3, given for each atomic number. The values are used only to compute the total number of core electrons in the molecule.

Note

The index in the example, 280 for Ni (Z=28), suggests the layout `core(10*Z)=...`. No source states this layout, so it must be verified.

Example:

```
$atoms core(280)=2,1,0,0 $end
# 1s, 2s, 2p (10 electrons) of nickel are core
```

See also: *core ($scf)*, *mcore ($atoms)*

5.9.2 mcore

Type: integer
Default: *none stated*
Status: current · not verified
Sources: input.txt

A simple way to add outer-core shells to the correlation treatment: `mcore=M` adds all shells whose principal quantum number is lower than that of the outermost valence shell by less than M.

See also: *core ($atoms)*

5.10 $optimize

Geometry optimization, saddle-point search, IRC and relaxed scans.

Relevant when: `task = optimize | optimize+hessian | irc | scan | opt_n1`

5.10.1 saddle

Type: switch
Default: 0
Status: current · not verified
Sources: input.txt; Dft.rtf; 04 #216; 03 §9

Type of stationary point sought.

Value	Meaning
0	Optimize to a minimum. (**default**)
1	Optimize to a saddle point.

Note

A saddle-point search needs an accurate Hessian in the input: compute one with `task=hessian` and append its `$Energy` block (the `eng>` lines). A usual route is scan → Hessian at the scan maximum → `saddle=1` → `task=hessian` to confirm one imaginary frequency → IRC.

See also: *follow ($optimize)*, *H ($Energy)*

5.10.2 tolerance

Type: list
Default: 0.0001,0.01
Units: au
Status: current · not verified
Sources: input.txt; 01 §4.6 (output echo)

Default: if only the first value is given, the second is 100 times larger (the recommended setting).

Convergence tolerance on the gradient (first value) and on the displacement (second value).

See also: *accuracy ($grid)*

5.10.3 trust

Type: list
Default: 0.2,0.0125
Status: current · not verified
Sources: input.txt; 01 §4.6; 04 #319; Dft.rtf

Default: echoed by v6, v13 and v15 as `Trust radius = 0.2 (max), 0.0125 (min)` in internal coordinates, and 0.1 / 0.00625 with `cartesian=1`. If only the maximum is given, the minimum is 1/16 of it and the initial value 1/4. `Dft.rtf` gives 0.5 (older version).

Trust radius (the largest allowed step), given as maximum, minimum and initial values.

5.10.4 follow

Type: integer
Default: *none stated*
Status: current · not verified
Sources: input.txt; Dft.rtf

Default: `1` according to `Dft.rtf`.

Follow the n-th Hessian eigenvector in a saddle-point search.

See also: *saddle ($optimize)*

5.10.5 steps

Type: integer
Default: `50`
Status: current · not verified
Sources: input.txt; Dft.rtf; 03 §8 (step/steps bug)

Maximum number of optimization cycles.

Note

If an optimization has not converged within about 50 steps, something is probably wrong; raising `steps` rarely helps. Computing a Hessian first and supplying it often does. In v6, if both `steps` and `step` are given, the value of `step` is used; give only one of them.

See also: *tolerance ($optimize)*, *steps ($d2edr2)*

5.10.6 cartesian

Type: switch
Default: `0`
Status: current · not verified
Sources: input.txt; 01 §4.6; 04 #518

Coordinate system for the optimization. The output reports `Coordinates: internal` or `cartesian`.

Value	Meaning
`0`	Redundant internal coordinates. (**default**)
`1`	Cartesian coordinates (last resort).

Note

`cartesian=1` is the fix when the program crashes with `process 0 got signal 6` after a `too few` message in the internal-coordinate table.

5.10.7 back

Type: switch
Default: `0`
Status: current · not verified
Sources: input.txt; 02 §3.2; 04 §0

IRC direction. A full IRC takes two runs, one per direction. Re-optimize the endpoints afterwards, because an IRC does not end at a converged minimum.

Value	Meaning
0	Trace the IRC path forwards. (**default**)
1	Trace the IRC path backwards.

See also: *points ($optimize)*, *radius ($optimize)*

5.10.8 points

Type: integer
Default: *none stated*
Status: current · not verified
Sources: input.txt

Number of points to optimize in an IRC run, or number of constrained geometry optimizations in a scan run.

See also: *value ($optimize)*

5.10.9 radius

Type: real
Default: *none stated*
Status: current · not verified
Sources: input.txt; Dft.rtf; 04 #298

Default: `0.2` according to `Dft.rtf` (older version).

Radius, in mass-weighted coordinates, for IRC path optimization. Adjacent IRC points end up roughly twice this distance apart. The IRC coordinate is reported in bohr·amuˆ1/2.

Note

`Vicinity of a Stationary Point(?) --- STOP.` during an IRC means the energy did not decrease on that step. Changing the step size is the reported fix.

5.10.10 coordinates

Type: list
Default: *none stated*
Status: current · not verified
Sources: input.txt; 03 §5 (type 4)

Default: generated automatically.

List of redundant internal coordinates to use, each given as five integers.

Value	Meaning
1,i,j,0,0	Distance i–j.
2,i,j,k,0	Angle i–j–k.
3,i,j,k,l	Torsion i–j–k–l.
4,i,j,k,l	Special coordinate for linear fragments. A user describes it as the angle between line l–k and the normal to plane k–j–i.
5,i,j,k,l	Special coordinate for linear fragments (meaning not described).

Note

`input.txt` spells this keyword `coordinated=` (apparently a typo). Verify the accepted spelling.

See also: *fix ($optimize)*

5.10.11 `fix`

Type: list
Default: *none stated*
Status: current · not verified
Sources: input.txt; 02 §3.2; 03 §5 (wish list); 04 #372-#373

Internal coordinates to keep fixed during the optimization, in the same five-integer format as `coordinates`. Several entries are separated by commas or spaces.

Note

Atoms cannot be frozen in Cartesian space; only internal coordinates can be fixed. Two coordinates also cannot be constrained to stay equal.

See also: *value ($optimize)*, *coordinates ($optimize)*

5.10.12 `value`

Type: indexed-array
Default: *none stated*
Units: Å and degrees
Status: current · not verified
Sources: input.txt; 03 §5, §8; 04 #278-#279, #373

Default: computed from the input geometry.

Values of the fixed internal coordinates. For `task=scan`, give all the initial values followed by all the final values. `val(i)=` fills the list starting at position i, so values left unset keep their input-geometry values.

Note

Priroda's dihedral angles have the opposite sign to the convention used by Molden, MOPAC and GAMESS. Flip the sign of any torsion value taken from another program, or the scan starts from a mangled geometry.

Example:

```
$optimize
 fix   =1,7,9,0,0, 1,8,10,0,0, 3,8,7,9,0, 3,10,9,7,0
 val(1)=1.6,       1.7
 val(5)=2.0,       2.1
 points=5
$end
# 5 optimizations: r(7-9) = 1.6 ... 2.0, r(8-10) = 1.7 ... 2.1;
# the last two fixed coordinates keep their input values
```

See also: *fix ($optimize)*, *points ($optimize)*

5.10.13 gradient

Type: switch
Default: *none stated*
Status: current · not verified
Sources: input.txt

Set `gradient=0` when the starting second derivatives were computed with a different method or basis set. This excludes the gradient terms from the internal/Cartesian transformation at the first geometry. Otherwise keep the default.

5.10.14 print

Type: switch
Default: *none stated*
Status: current · not verified
Sources: input.txt

Amount of optimization output.

Value	Meaning
0	Minimal.
1	Detailed

5.10.15 sym

Type: switch
Default: *none stated*
Status: undocumented · not verified
Sources: 01 §4.6; 04 #481

Used as `sym=1` in the official 2023–2025 example inputs, apparently to preserve molecular symmetry during the optimization.

See also: *sym ($molecule)*

5.10.16 mod

Type: integer
Default: *none stated*
Status: undocumented · not verified
Sources: 01 §4.6

Used as `mod=1` with `task=opt_n1` (very large systems). Meaning not documented.

5.11 $d2edr2

Numerical Hessian by finite differences of the gradient.

5.11.1 length

Type: real
Default: *none stated*
Status: current · not verified
Sources: input.txt

Step length for numerical differentiation of the potential-energy surface.

5.11.2 displace

Type: switch
Default: 2
Status: current · not verified
Sources: input.txt

Number of displacements per coordinate. More accurate numerical Hessians may also need tighter SCF convergence.

Value	Meaning
1	One displacement per coordinate.
2	Two displacements per coordinate. More accurate but twice as slow. (**default**)

See also: *convergence ($scf)*

5.11.3 steps

Type: list
Default: 0,-1
Status: current · not verified
Sources: input.txt

Default: 0,-1 runs every step from 0 to 3N (or 6N) in a single run.

The first and last displacement steps to do in this run. The reference geometry is step 0, and the displaced geometries are 1 … 3N (one displacement) or 1 … 6N (two displacements). This lets a numerical Hessian be split across separate runs and then combined.

Note

Not the same keyword as $optimize steps.

Example:

```
# 12-atom molecule, four inputs run in parallel on different machines:
$d2edr2 disp=1 steps= 0, 9 $end    # mol.in1
$d2edr2 disp=1 steps=10,18 $end    # mol.in2
$d2edr2 disp=1 steps=19,27 $end    # mol.in3
$d2edr2 disp=1 steps=28,36 $end    # mol.in4
# collect the partial results into one input:
grep "num>" mol.out{1,2,3,4} | sed "s/\.*num>//g" >> mol.in
# add the line below and run once more to process them:
$d2edr2 steps=-1 $end
```

5.12 $thermo

Thermochemistry after a Hessian calculation.

5.12.1 sigma

Type: integer
Default: *none stated*
Status: current · not verified
Sources: input.txt

Rotational symmetry number for thermochemistry.

5.12.2 temperature

Type: list
Default: *none stated*
Units: K
Status: current · not verified
Sources: input.txt; Dft.rtf

Default: `298.15` according to `Dft.rtf`, but posted outputs show `298.000 K` (2011, 2021) as well as `298.150 K` (2013). Set `temperature=` explicitly when the exact value matters.

Temperatures at which thermochemistry is printed, up to 50 values.

5.12.3 mass

Type: list
Default: *none stated*
Units: amu
Status: undocumented · not verified
Sources: Dft.rtf

Atomic masses for vibrational analysis and thermochemistry. Use it to redo the analysis for other isotopes from a stored Hessian (`$Energy`) without recomputing it.

See also: *mass ($molecule)*, *H ($Energy)*

5.13 $nmr

Reference shieldings and extra points for NMR shielding calculations.

Relevant when: `task = nmr (theory = dft only)`

5.13.1 standard

Type: indexed-array
Default: *none stated*
Status: current · not verified
Sources: input.txt; 04 #69; 02 §5.7

Reference shielding constants, indexed by atomic number, for computing chemical shifts. Values are needed only for the elements present in the molecule.

Note

A reference shielding is only valid for the method, basis set and grid it was computed with. Users quote different TMS values (e.g. `standard(1)=31.3867 standard(6)=181.9781`) for other settings. Compute TMS with exactly the settings of the production run.

Example:

```
$nmr
 standard(1)=31.3915
 standard(6)=181.9921
$end
# computed for Si(CH3)4; usable for a series of hydrocarbons
```

5.13.2 points

Type: list
Default: *none stated*
Units: Å
Status: current · not verified
Sources: input.txt; 02 §3.2; 03 §12

Extra points in space, given as x,y,z triples, at which magnetic shielding tensors are computed in addition to the nuclear centres. This is the way to compute NICS: NICS is the negative of the isotropic shielding printed for the point.

Example:

```
$nmr
 points=
      0.00000000    0.00000000    0.00000000
      0.00000000    0.00000000    1.00000000
$end
```

5.14 $Energy

Results of an earlier run (energy, gradient, Hessian, dipole data), read back as input.

The program prints this group on lines prefixed with `eng>` (an energy/gradient/Hessian run), or `num>` (the pieces of a distributed numerical Hessian; see `$d2edr2 steps`). Strip the prefix and append the group to the next input. The shipped helper script `ge` does this.

A stored Hessian is required for `$optimize saddle=1` and `task=irc`. It also speeds up difficult minimizations, and it lets you redo vibrational analysis or thermochemistry (for example for other isotopes, `$thermo mass=`) without recomputing.

```
eng>$Energy
eng> E=-2.284057276940e+03
eng> D= 4.22290114e-08 -1.82061636e-08  2.32661117e-08
eng> G= 5.13271929e-06 -5.52234429e-06 -6.15997899e-06
eng>    ...
eng>$end
```

5.14.1 E

Type: real
Default: *none stated*
Units: hartree
Status: undocumented · not verified
Sources: 01 §4.10; 02 §3.2

Total energy.

5.14.2 G

Type: list
Default: *none stated*
Units: au
Status: undocumented · not verified
Sources: 01 §4.10; 02 §3.2

Cartesian gradient (3N values).

5.14.3 H

Type: list
Default: *none stated*
Units: au
Status: undocumented · not verified
Sources: 02 §3.2

Hessian as a lower triangle, 3N(3N+1)/2 values.

See also: *saddle ($optimize)*

5.14.4 D

Type: list
Default: *none stated*
Units: au
Status: undocumented · not verified
Sources: 01 §4.10; 02 §3.2 (inferred)

Dipole moment.

5.14.5 Dx

Type: list
Default: *none stated*
Status: undocumented · not verified
Sources: 02 §3.2 (inferred)

Dipole derivatives with respect to nuclear coordinates (inferred from use: IR/Raman intensities).

5.14.6 A

Type: list
Default: *none stated*
Status: undocumented · not verified
Sources: 02 §3.2 (inferred); 04 #381

Polarizability, and its derivatives after `task=dipole` (inferred; used for Raman intensities).

5.15 $dynamic

Molecular dynamics with a thermostat.

Relevant when: `task = thermostat`

Known only from the official example inputs and a third-party input generator. Every meaning below is inferred, except `file`.

```
$dynamic
 e=0.001
 d=0.03
 n=1000,100
 m=10
 i=1
 file=H40C20/H40C20_%06d.qm.in
$end
```

5.15.1 e

Type: real
Default: *none stated*
Units: hartree
Status: undocumented · not verified
Sources: 01 §4.8; 02 §3.2

Thermostat temperature as kT (inferred: a generator script sets e = T/315775).

5.15.2 d

Type: real
Default: *none stated*
Status: undocumented · not verified
Sources: 01 §4.8; 02 §3.2

Time step (inferred).

5.15.3 n

Type: list
Default: *none stated*
Status: undocumented · not verified
Sources: 01 §4.8; 02 §3.2

Number of steps and print interval (inferred).

5.15.4 m

Type: integer
Default: *none stated*
Status: undocumented · not verified
Sources: 01 §4.8; 02 §3.2

Meaning not known.

5.15.5 i

Type: integer
Default: *none stated*
Status: undocumented · not verified
Sources: 01 §4.8; 02 §3.2

Run index or random seed (inferred).

5.15.6 v

Type: real
Default: *none stated*
Units: bohr^3 per atom
Status: undocumented · not verified
Sources: 01 §4.8; 02 §3.2

Volume per atom of a spherical confining cavity, e.g. for simulating a liquid (inferred).

5.15.7 h

Type: real
Default: *none stated*
Status: undocumented · not verified
Sources: 01 §4.8; 02 §3.2

Stiffness of the cavity wall (inferred).

5.15.8 file

Type: string
Default: *none stated*
Status: undocumented · not verified
Sources: 01 §4.8

printf-style pattern for snapshot files, which are written as ready-to-run inputs; `%06d` is the frame number.

5.16 $uniq

Duplicate-structure filtering over several `$molecule` blocks.

Relevant when: `task = uniq`

5.16.1 dist

Type: real
Default: *none stated*
Status: undocumented · not verified
Sources: 01 §4.9

Threshold for treating two structures as identical (inferred from the official example `dist=0.01`, run on 157 MD snapshots, each in its own `$molecule` block).

CHAPTER

SIX

SOLVING COMMON PROBLEMS

This chapter is arranged by symptom. Each entry names the likely cause and the fix that users have reported.

Almost everything here comes from user forums, not from the developer. Each fix says how well it is supported: "confirmed by several users" means more than one person reported that the fix worked, and "single report" means one user posted it and nobody confirmed it. Many reports date from Priroda 6 to 10 (2009–2015). Where a problem may have been fixed in later builds, the entry says so.

Note

None of the fixes below has yet been checked against a current Priroda binary.

6.1 The input is not read, or is read wrongly

When Priroda cannot parse the input it usually stops early, often right after echoing `molecule input: ''`, and gives no explanation. Before anything else, open the output and check what it echoed back: the `System options` block, the `scf options:` line, the `grid:` line and the summary box (number of atoms, charge, multiplicity, method, functional). If a value you set is not shown there, Priroda did not read it.

6.1.1 Output stops after `molecule input: ''`

Cause. In almost every reported case the input file had Windows (CRLF) line endings. The tail of the output then shows a mangled keyword followed by `?`, for example `' ?matrix`, `?rtesian` or `'z-matrix' ?`. Inputs saved as Unicode, or edited in Word, fail the same way. Word's "save as text" still writes CRLF line endings.

Fix. Save the input as plain ASCII text with Unix (LF) line endings, for example with `dos2unix input.in`. Notepad++ and Far Manager (Shift+F2 in the editor) can also convert line endings. Use straight quotes only. Several users confirmed this fix. Running `dos2unix` on a file that already has LF endings does no harm.

A second cause gives the same first line: atomic numbers written as real numbers (`25.0` instead of `25`) in `$molecule`. In that case the output continues with `illegal charge/multiplicity`. Write atomic numbers as integers. This fix was posted by one experienced user.

Avoid Cyrillic letters and spaces in the computer name, directory names and file names. One user's problem with this output went away after he had dealt with both line endings and such names.

6.1.2 `task=scan` runs an optimization, or nothing works on one machine

An input that runs on one computer but not on another, or a `task=scan` that silently runs a plain optimization, has been traced to the same encoding problem: an input edited in Word or saved as Unicode. Fix it as above.

Two `task=` lines in one input do not chain jobs. Only the first one runs, and no error is printed. The only combined task that works is `task=optimize+hessian`; see *task*. Run other sequences, such as optimization followed by NMR, as separate jobs.

6.1.3 The basis file is not found

- Quote file names with straight double quotes: `basis="3z.bas"`. Two single quotes (`basis=''3z.bas''`) give `error opening file ''3z.bas''`, and typographic quotes copied from a web page or word processor fail too.
- Give an absolute path to the basis file if Priroda is not started from the directory that contains it. One user's parallel run aborted with `MPI_Abort` until he wrote the full basis path in the input.
- The relativistic basis file `basis4.in` needs *four=1* in *$control*. Without it the job stops at the `Basis set input:` line. The user who reported this confirmed the fix.
- The name given in *set* must exist in the basis file.

6.1.4 The input or output file cannot be opened

On Linux the usual cause is a wrong path. Check it with `ls -l` before blaming Priroda. In one reported case the directory in the command line simply did not exist. Work in a directory you own and can write to (for example `~/priroda`), `cd` into it, and run the binary with an explicit path, since the current directory is normally not in `PATH`:

```
cd ~/priroda
./p job.in job.out
```

File names are case-sensitive, unlike keywords. One user saw the extra complaint `0:  output file 'tasK.out'?` when the file name contained capital letters. Using lower-case names for input and output avoids it; this is a single observation.

On Windows, `cd` into the folder with the input before running `p.exe input.in output.out`. One user still got `cannot open input file` after doing that, and the thread never found the cause.

6.1.5 Output stops at `0:  opening file /tmp/...` (Windows)

Old Windows builds put scratch files in `/tmp` when nothing else is set (the output echoes `Disk:  1 GB '/tmp'`), and Windows has no `/tmp`. Set the scratch directory explicitly, for example `$system path=. $end`, or set the `TMPDIR` environment variable before starting the program. This is the oldest known problem in both forums and the fix is well established. See *path*.

6.1.6 `$scf` settings are ignored or rejected

Two problems are reported when *convergence* and *iterations* appear together in *$scf*:

- The parser stops with `*** error in '$scf' :  unknown name erations`, or the rest of the group is silently skipped. One user found that the order of the two keywords mattered; another got the error in either order.
- In a different test, the pair worked only when *procedure=bfgs* was also given.

Warning

These reports conflict and come from 2015 builds. Until this is checked on a current build, put `iterations` before `convergence` on separate lines, and confirm both values in the `scf options:` line of the output, for example `scf options:  conv=1.0e-07, 1.0e-03; iter=250,16`.

6.2 SCF does not converge or diverges

6.2.1 Recognizing the problem

Users report that Priroda's SCF usually converges well, better than in many other programs. Trouble mostly comes with metals in low oxidation states, where the electronic structure has some multiconfigurational character, with very large basis sets, and with diffuse functions.

Signs of trouble in the SCF table:

- `hmin` values that keep growing in magnitude, or a line such as `h = -25.15!`;
- `lowest eigenvalue = ...!!!` lines with large negative numbers;
- an energy that rises from one macro-iteration to the next;
- `no convergence!` followed by `SCF is far from convergence`, after which Priroda stops.

The DFT SCF runs on a coarse grid first and then switches to the final grid; the switch is marked by a new `Grid:  ...  points` line. A short jump in the gradient at that point is normal. Steady growth after it is not (see *Divergence after the switch to the fine grid (open-shell transition-metal complex)*).

6.2.2 Check the input first

In several reported failures the SCF was not the real problem. Before changing SCF settings, check:

- **Multiplicity.** *mult* must match the electron count. One failing input gave `mult=0` for an odd number of electrons, and the summary box showed `Spin multiplicity 0`.
- **Theory and functional.** A hybrid functional with `theory=dft` is wrong; see *Hybrid functional silently ignored*.
- **Basis.** Non-native basis sets and diffuse functions converge badly. Priroda's auxiliary (fitting) bases are not designed for foreign orbital bases. Use a native basis, or start from a geometry optimized in one.
- **Geometry.** A very poor starting geometry can by itself prevent SCF convergence. Pre-optimize with a cheap pure functional (for example PBE with `theory=dft`) before switching to the target method.

6.2.3 Restart from saved vectors

This is the technique most often recommended by experienced users, on both forums, and the limor1 `pri` driver script automates it:

1. Cap the number of SCF iterations at a moderate value, for example `iterations=10` in *$scf*. Choose the cap so that `hmin` does not run away.
2. Have Priroda write its vectors with *save* in *$control*.
3. When the run stops with `SCF is far from convergence`, add `$guess read=1 $end` (see *read*) and rerun with the latest geometry.
4. Repeat until the SCF converges.

```
$control
 task=optimize
 theory=dft
 basis="basis.in"
 save=job.vec
$end
$scf iterations=10 $end
$guess read=1 $end
```

Leave out the `$guess` line in the first run. Vectors from a converged run of an easier case, or of the same molecule at another geometry, also work as a starting point.

6.2.4 Switch the SCF procedure and tighten the grid

The default *procedure* is Newton–Raphson (`nr`). If it blows up, try `bfgs`, and the other way round. One experienced user recommends this setting for difficult cases, "which is nearly all of them":

```
$scf
 convergence=1e-7
 procedure=bfgs
 iterations=250
$end
$grid
 accuracy=1e-9
$end
```

He notes that the default 16 iterations are too few for many jobs. If you tighten *convergence* to `1e-8`, make the grid finer as well, `1e-9` or `1e-10` (see *accuracy*).

Note

The user who asked never reported whether this fixed the failing job. `bfgs` is not a cure-all: in the case in the next section it may have been part of the problem.

6.2.5 Divergence after the switch to the fine grid (open-shell transition-metal complex)

Symptom. A doublet Cu(II) complex (OLYP/L2, `restrict=0`, `procedure=BFGS`, `$grid acc=1e-12`, `task=hessian`) converged on the coarse grid. After the switch to the final grid it oscillated for about 30 iterations and then diverged, with the total energy reaching +29 hartree. BLYP behaved the same way.

Fixes. Two experienced users solved it in two ways, which they considered equivalent:

- Run `task=optimize+hessian` instead of a single-point Hessian, and drop `procedure=BFGS` and `iterations`:

  ```
  $control task=optimize+hessian $end
  $scf restrict=0 convergence=1e-10 $end
  ```

  Each optimization step then starts from the previous wavefunction.

- Save the wavefunction at the start of the optimization with *save* and read it into the Hessian job with `$guess read=1`. For an already optimized structure, the guess built from scratch turned out to be poor.

6.2.6 Open-shell singlets and broken symmetry

For an open-shell (broken-symmetry) singlet, use an unrestricted calculation and mix the HOMO and LUMO of the guess:

```
$scf restrict=0 $end
$guess mix=1 $end
```

See *restrict* and *mix*. With non-RI pure DFT (`theory=dft`, PBE or BLYP), one user found that `restrict=0 mix=1` converged back to the restricted solution, with identical energies and identical alpha and beta orbital energies. RI-DFT broke the symmetry in his case. Another user notes that pure GGA functionals often do not break symmetry, and suggests a hybrid (with `theory=ridft`) or a Hartree–Fock calculation first.

6.2.7 Heavy elements

For scalar-relativistic pure DFT with the heaviest atoms, the v5 documentation suggests *d1small=1*; without it, the quadratically convergent SCF may fail.

If an orbital ordering problem is suspected, *na* and *nb* reorder the starting vectors.

6.2.8 Cases without a known fix

- An HF reference for CCSD diverged for a small closed-shell molecule (7 atoms, `set=L1`, `restrict=0`). The `change` column stuck at `0.50000000`. Nobody replied. Running a closed-shell molecule with `restrict=0` was unusual and may be worth changing.
- An HfCl2 molecule (PBE, `four=1`, `basis4.in`, `set=L3`, `accuracy=1e-10`) would not converge. Nobody replied.

6.3 Geometry optimization problems

6.3.1 The optimization does not converge within `steps`

The default limit is 50 steps (see *steps*). The documentation's advice is that an optimization still running after about 50 steps usually has some other problem, and raising `steps` rarely helps. What users do instead:

- **Restart from the last geometry.** The limor1 `pri` script restarts Priroda from the last geometry whenever an optimization runs out of steps. This helps when the optimizer cycles around one geometry even after 300 steps.
- **Supply a Hessian.** Run `task=hessian`, append its `$Energy` block (the `eng>` lines) to the input, and optimize again. See *$Energy*.
- **Use Cartesian coordinates.** *cartesian=1* is described in the documentation as a last resort, but users find it more robust for automated runs and for floppy systems such as explicit solvation shells.

Users also say that Priroda's optimizer is weaker than Gaussian's, while its SCF is stronger.

6.3.2 Crash with `process 0 got signal 6` and `too few`

Symptom. The optimization dies with `process 0 got signal 6`. In the output of the last step, the `INTERNAL COORDINATES` table ends with `too few`.

Fix. Optimize in Cartesian coordinates:

```
$optimize cartesian=1 $end
```

This was the answer from one experienced user; the asker did not report back.

6.3.3 The optimization "converges" in one step

A run that reports convergence after a single step, with an energy that is not a number, did not optimize anything: the SCF failed on the first step. See *NaN energy, or an optimization that stops after one step*.

6.3.4 A symmetric structure stays symmetric

Priroda always works in C1, but a symmetric starting geometry often stays symmetric through an optimization that converges normally, and the Hessian then shows a saddle point. If the optimization did not break the symmetry, always compute the Hessian. If it has an imaginary frequency, paste its `$Energy` block into the input, displace the geometry along that mode if needed, and optimize again.

6.3.5 A scan starts from a mangled geometry

Cause. Priroda's dihedral angles have the opposite sign to the convention used by Molden, MOPAC, GAMESS and most other programs. A dihedral *value* copied from another program makes the first scan point a scrambled structure that has nothing to do with the input geometry.

Fix. Flip the sign of every dihedral value taken from another program. The user who hit this confirmed that flipping the signs fixed it, and the sign difference is reported independently on the other forum.

A scan that requested 10 points but computed only 7 (and 11 of 20) was reported once and never explained.

6.3.6 IRC gives only two points, or no curve

An IRC must start from a properly optimized transition state with an accurate Hessian in the input. Find the TS first (`task=optimize` with *saddle=1*), compute the Hessian there, check that it has exactly one imaginary frequency and that this mode is the reaction you want, and only then run `task=irc`. An IRC started from an unoptimized geometry gives two points with *back=0* and no curve with `back=1`.

6.3.7 IRC stops with `Vicinity of a Stationary Point(?) --- STOP.`

On the first IRC step this means that the energy did not go down on that step. The reported fix is to change the step size; see *radius*:

- on a very flat barrier top, increase the step so that it reaches the slope;
- on a very narrow barrier, such as a proton transfer, decrease it, because a step suited to heavy atoms throws the proton too far.

This advice comes from one experienced user.

6.3.8 IRC ends near a minimum but not in it

This is normal: always re-optimize both IRC endpoints. An IRC can also stop on a flat shelf where the gradient is small, or at another TS. Then compute a Hessian at the stopping point and run a new IRC from there; this sometimes has to be repeated three or four times. Priroda has no option to recompute the Hessian every n steps.

6.4 Wrong or suspicious results

6.4.1 Hybrid functional silently ignored

Cause. Hybrid functionals (B3LYP, PBE0/PBE1, ...) are applied only with *theory=ridft*. With `theory=dft` or `theory=tddft`, older versions (reported for v6 to v8) ran without any error message but used zero exact exchange, so the result was a pure-GGA-like calculation labelled as a hybrid. The developer was told about it and, according to a user who corresponded with him, did not intend to change it.

Newer builds (reported in 2014) refuse instead: the job prints `Basis set input:` twice and stops with `application called MPI_Abort(MPI_COMM_WORLD, 1) - process 0`, before the summary box. Several users reproduced this.

Fix. Use `theory=ridft` for every hybrid functional. There is no hybrid TDDFT in Priroda.

6.4.2 NaN energy, or an optimization that stops after one step

Cause. The SCF energy on the first step is not a number, so the optimizer stops at once and reports convergence. The reported causes are all basis problems:

- **No basis for an element.** A molecule containing Bi was run with `3z.bas`, which covers only Z ≤ 54. Use `sbk.bas` (ECPs on every element) or `basis4.in` with *four=1*.
- **ECP not read.** In one case nothing was computed and there was no error message. The output showed zeros for one element in the basis table, and no sign that the ECP had been read. The cause was a defect in the user's basis file; `20+sbk.bas` worked.
- A NaN at the first SCF iteration for [PtCl6]2- with the St-rsc-97 ECP basis went away after switching to the `sbk(u)` basis. Why that basis failed is not known.

What to check. In the output, the ECP should be read before the basis, and every atom in the basis table should have a non-zero number of basis functions.

6.4.3 TDDFT oscillator strengths are all zero

With `restrict=0`, singlet and triplet excitations get mixed. Use `restrict=1` and compute one multiplicity per run. The user who reported the problem confirmed the fix.

6.4.4 The dipole moment is zero

Version 6 printed `dipole = 0.000000 au ( 0.000000 D)` for every molecule when relativistic basis sets were used, even with `task=dipole`. Version 8 and later print a value.

6.4.5 Energies differ between Priroda versions

Newer builds are faster but give slightly different total energies for the same input. Do not compare close energies computed with different versions; within one version the comparison is fine.

6.4.6 The printed energy components do not add up to E

The line `T = ... V = ... J = ... Ex = ... Ec = ...` does not include the nuclear repulsion energy, so the components do not sum to `E`. This is not an error: E = T + V + J + Ex + Ec + E(nn). The answer came from the developer, relayed by a user.

6.4.7 Thermochemistry of single atoms

For a monoatomic species the enthalpy has an extra RT, and the rotational entropy is printed as `-inf`. Correct these by hand and keep only the translational terms.

6.4.8 Results depend on the number of processes

In one unstable TS search started from a poor guess, the optimization followed different paths with different `-np`. Normally the results differ only in insignificant digits.

6.5 Memory, disk and scratch errors

All *$system* limits apply per MPI process. With `disk=10` on four processes, up to 40 GB can be used.

6.5.1 `malloc(...) failed`

Cause. Priroda could not allocate the memory requested in *memory*, plus any RAM disk requested with a negative *disk*. The number in the message is the request in bytes.

Fix. Ask for less than the node really has free:

- With a RAM disk, `memory` plus —`disk`— (both in MB), times the number of processes, must fit in physical RAM. An example from a 24 GB node is `memory=9000 disk=-14000`. Several users confirmed this rule.
- The 32-bit Windows (Cygwin) builds cannot allocate more than about 1 GB. 512 MB was enough for most DFT jobs. Several users confirmed this limit. The 64-bit Windows build with `cygwin1.dll` is reported to have no such limit.

A message beginning `We need 1.5 GB of memory...` was reported once. One reply guessed that it is about RAM (`memory=`), not disk space, but this was not confirmed.

6.5.2 `out of disk space`

Symptom. `0: out of disk space (VNNNN), <N>KB`, or a similar message naming another array, such as `(ci.Rnvvo)`.

Fix.

- Raise *disk* above the amount reported. Remember that the value is per process. One user's correlated Hessian that ran out of disk on 16 processes then ran with `disk=40` on one fast drive.
- Conventional MP2 is very disk-hungry. Use `theory=rimp2` instead.
- Check that `cartesian` or `z-matrix` in `$molecule` matches the coordinates you actually gave; wrong coordinates can make the job far bigger than intended.

Each scratch file is created for 1 GB per process and filled as needed. Jobs needing more than 20–45 GB of scratch were, in one experienced user's words, not worth their cost.

A failure to read a scratch file during MP2 (`0: read file ./tmp/...`) was given the same fix.

6.5.3 A small RI-MP2 job does not fit in memory

A 7-atom RI-MP2 job with `four=1`, `basis4.in` and `set=L22` claimed not to fit in memory however much was given, while smaller and larger basis sets ran. A second user reproduced it and got `0: write VMNN(11753704:14042536)?`. Both found that direct integrals avoid the problem:

```
$integral direct=1 $end
```

See *direct*.

6.5.4 Where scratch files go

Priroda writes scratch files to the directory given by *path*; if `path` is not set, to `$TMPDIR`; otherwise to the directory of the input file. On a cluster, neither `/tmp` nor the input directory is suitable for large scratch files: `/tmp` may sit on a quota-limited node. Use the cluster's local scratch area, or a RAM disk with a negative `disk` value. Never put scratch on NFS.

A job that stops while writing a scratch file, with a message like `0: write file /tmp/tmp9633.11 (leAX): fd=21 ...`, was reported once during an analytic Hessian and not answered. A full scratch directory or a file-size limit is a plausible cause, but this is a guess; check the free space and the `disk=` setting.

If Priroda does not get enough memory or scratch, it stops with an error; it does not fall back to a slower algorithm.

6.5.5 Slow runs: disk speed

Priroda is very demanding of disk speed. For all methods except GGA DFT, the documentation warns that performance degrades "terribly" if more than one process reads and writes large files on the same physical drive. Options, in order of preference:

- keep scratch in RAM with a negative *disk* value (one cluster reported a 6× speed-up for RI-MP2);
- give each process its own physical drive with a colon-separated *path* list;
- if the job is disk-bound, switch to direct integrals with *direct=1*. This costs CPU time but can be much faster overall: one RI-MP2 optimization went from 2 to 20 steps per night.

One user with 16 processes ignored the warning and put all scratch on one fast disk.

6.6 Parallel and MPI problems

6.6.1 A parallel RI job hangs with no message

Cause. RI methods, RI-MP2 and RI-DFT included, hang forever if any process gets no work. This happens when the job is too small for the number of processes. Two experienced users confirmed it and called it a bug.

How to recognize it. After `integral estimates`, the output lists the blocks per node. Look for a line with `0 blocks`:

```
   2892490 blocks,    103625770 integrals on node  0
   3681560 blocks,     91256706 integrals on node  1
```

Fix. Run small RI jobs on fewer processes, or on one core.

Older reports describe similar hangs: small jobs on many processors hang (v6/v7); MP2 on a 102-atom complex died with signal 8 on more than about 24 cores (v7, MVAPICH); a v6 Hessian on 4 cores could hang at `Solving for responses ...`. The v6 advice was at most 2 cores for a Hessian and at most 4 for an optimization. Another user ran DFT, including Hessians, on 24–32 cores without trouble, so this advice is disputed. Parallel runs work properly only from version 16 on.

6.6.2 Little or no speed-up from more cores

- DFT scales well within one node but gains little beyond 6–8 cores: one user measured at most 10% more from 32 cores than from 6–8. Another measured 101, 63 and 37 s for the same optimization on 8, 12 and 24 cores.
- Running across several nodes gives no gain. Several experienced users report this.
- In a DFT run most of the memory and all the disk I/O stay on rank 0, so spreading a DFT job over nodes does not spread its memory. RI-MP2 distributes memory better.
- For throughput, several single-core jobs at once can beat one multi-core job when the jobs are not disk-bound.

- For RI-MP2, the v5 documentation suggests a number of processes that divides the number of occupied orbitals.

6.6.3 Starting a parallel run on Linux

The Priroda binary starts its own processes: run `./p -np 4 job.in job.out`. Starting it through the bundled `mpiexec` failed with `Error in system call mpiexec: No such file or directory` or `mpiexec could not exec p`. Use absolute paths for the binary, the input, the output and the basis file.

The message `OOPS: semop lock failed` (with `process N received signal 2` or 6) has two reported contexts:

- as a secondary message after another fatal error, such as running out of disk. Fix the first error;
- in v6 shared-memory runs on 7 cores. Fewer cores avoided it; cleaning semaphores with `ipcrm` did not help.

It also appeared for one user who ran as a normal user while the same input worked as root. That case was never resolved.

A run that stops producing output right after the header, while the process keeps using CPU, has been reported and called common. The only advice was to look for stray files named after the input or output in the input, output, program and home directories, and then to rule out causes one by one.

6.6.4 Windows

Old Windows builds were serial only. For a parallel 64-bit run (version 16 or later), users report this setup:

1. From a 64-bit Cygwin installation with OpenMPI, copy `cygwin1.dll`, `cyggcc_s-seh-1.dll`, `cygmpi-12.dll`, `cygopen-pal-13.dll`, `cygopen-rte-12.dll` and `orterun.exe` next to the Priroda executable. Take `cygwin1.dll` from the same Cygwin installation, because the other DLLs work only with it. These are the names from a 2017 Cygwin. A 2018 report says newer OpenMPI packages ship `cygopen-pal-40.dll` and `cygopen-rte-40.dll` instead; take those.
2. To run outside the Cygwin terminal, create a `dev` directory one level above the directory you start Priroda from, with subdirectories `shm` and `mqueue`. For Priroda in `C:\Priroda 16` that means `C:\dev\shm` and `C:\dev\mqueue`. Without them Cygwin prints `Warning: '/dev/shm' does not exists or is not a directory.`
3. Set `TMPDIR` and start the run from `cmd.exe`:

```
cd /d "C:\Priroda 16"
set TMPDIR="%TEMP%"
"orterun.exe" -np 2 ./p16_64_parallel.exe "C:\Work\file.in" "C:\Work\file.out"
```

4. If the program is in a system directory such as `C:\Program Files`, run it as administrator.

Network problems reported with this setup:

- On a PC connected by WiFi that also has a LAN adapter, OpenMPI failed with `tcp_peer_send_blocking: send() to socket 12 failed`. Disabling the LAN adapter fixed it (single report).
- With the Ethernet cable unplugged, only some cores were used. Plugging it back in fixed it (single report).

A 2021 attempt to follow this recipe on Windows 10 failed with an error that was posted only as a screenshot; no fix was posted. One user runs Priroda under WSL on Windows instead; no details of that setup were posted.

Warning

Not every build on the official site is reliable. A note there once said that the `w64/*.exe` 64-bit Cygwin builds had a DFT/ECP bug. If a Windows build misbehaves, ask the developer which build you should be running.

CHAPTER

SEVEN

SOURCES

This manual is not an official Priroda document. It was not written or reviewed by the developer of Priroda, Dimitri Laikov. It is compiled from the sources below, which are listed roughly in the order of how much weight the manual gives them when they disagree. Each keyword entry in the *input reference* names the sources it rests on.

7.1 Program files and text from the developer

Official file index
: https://rad.chem.msu.ru/~laikov/p_w/, read in October 2026 (page dated June 2025). A bare listing of executables, example inputs and outputs, basis-set and parameter files, and the helper scripts `ge`, `gm` and `grx`. It has no prose documentation. Example outputs from this site are the strongest evidence for defaults and output formats. Windows builds are in the `exe64/` subdirectory.

Input Summary (`input.txt`)
: An English keyword list shipped with the program, apparently written by the developer, with user annotations appended. It is the closest thing to an official keyword reference. A few of its Russian annotations are corrupted and cannot be read.

Version 5 keyword reference (KNC wiki mirror)
: The developer's "Input Summary" for version 5, mirrored on the wiki of the Kazan Scientific Centre computing centre together with a beginner FAQ. The wiki is offline; pages were recovered from the Internet Archive (captures 2008–2012). A PDF copy of the documentation page is at https://monitor.hpc.umanitoba.ca/doc/Priroda_Documentation_from_KNCWiki.pdf.

`Dft.rtf`
: A short Russian keyword list with practical advice, from an older version. Some of its defaults differ from later versions.

`Dft_ansi.txt`
: A Russian description of the input of Priroda's predecessor. Its group names are obsolete; it is used here only for history.

7.2 User-written material

Forum statements attributed to the developer reach these sources only second-hand, through users who relayed his e-mails. The manual marks such statements as relayed.

chemport.ru forum, thread 28803 ("Priroda")
: https://chemport.ru/forum/viewtopic.php?f=71&t=28803. The main Russian-language user thread, 518 posts from 2009 to 2026. The largest single source for practical advice and known problems.

chemport.ru forum, thread 112860 (“Difficult cases with Priroda”)
https://chemport.ru/forum/viewtopic.php?f=71&t=112860. Six posts (2015–2016) on SCF divergence, RI-MP2 memory and parallel hangs, with complete inputs and outputs.

qchem.ru forum, thread “Quantum-chemical program Priroda”
The forum of the Ufa Quantum Chemistry Society, 2007–2012. The site has been offline since about 2019; posts up to August 2012 were recovered from the Internet Archive.

LIMOR page, Novosibirsk Institute of Organic Chemistry
http://limor1.nioch.nsc.ru/priroda.html. Example inputs, tables of basis-set names, and Perl helper scripts (`pri`, `priin`, `pri2mol` and others) written by users. Much of what the manual says about the output format is inferred from how these scripts parse it.

7.3 Published papers

The papers listed under *How to cite* describe the methods behind Priroda. They are cited where the manual explains a method, not as documentation of the input.

INDEX

Symbols

$Energy, 52
$atoms, 44
$control, 32
$d2edr2, 49
$dft, 41
$dynamic, 54
$grid, 42
$guess, 38
$integral, 43
$molecule, 36
$nmr, 51
$optimize, 44
$scf, 39
$system, 30
$thermo, 51
$uniq, 55

A

A (*$Energy*), 53
accuracy (*$grid*), 42

B

back (*$optimize*), 46
basis (*$control*), 34

C

cartesian (*$optimize*), 46
charge (*$molecule*), 36
convergence (*$scf*), 40
coordinates (*$optimize*), 47
core (*$atoms*), 44
core (*$scf*), 41

D

d (*$dynamic*), 54
D (*$Energy*), 53
d1small (*$scf*), 41
direct (*$integral*), 43
disk (*$system*), 31
displace (*$d2edr2*), 50
dist (*$uniq*), 55
Dx (*$Energy*), 53

E

e (*$dynamic*), 54
E (*$Energy*), 53

F

file (*$dynamic*), 55
fix (*$optimize*), 48
follow (*$optimize*), 45
four (*$control*), 34
functional (*$dft*), 41

G

G (*$Energy*), 53
gradient (*$optimize*), 49

H

h (*$dynamic*), 55
H (*$Energy*), 53

I

i (*$dynamic*), 55
iterations (*$scf*), 40

L

length (*$d2edr2*), 50

M

m (*$dynamic*), 54
mass (*$molecule*), 37
mass (*$thermo*), 51
mcore (*$atoms*), 44
memory (*$system*), 30
mix (*$guess*), 39
mod (*$optimize*), 49
mult (*$molecule*), 37

N

n (*$dynamic*), 54

na ($guess), 38
nb ($guess), 39
nucleus ($control), 35

P

path ($system), 31
points ($nmr), 52
points ($optimize), 47
print ($control), 35
print ($optimize), 49
procedure ($scf), 40

R

radius ($optimize), 47
read ($guess), 38
restrict ($scf), 39
routine ($integral), 43

S

saddle ($optimize), 45
save ($control), 36
set ($molecule), 37
sigma ($thermo), 51
standard ($nmr), 51
state ($control), 34
steps ($d2edr2), 50
steps ($optimize), 46
sym ($molecule), 38
sym ($optimize), 49

T

task ($control), 32
temperature ($thermo), 51
theory ($control), 33
tolerance ($optimize), 45
trust ($optimize), 45

U

unit ($molecule), 37

V

v ($dynamic), 55
value ($optimize), 48